# Project Discovery and Hierarchical Data

Kirill Nevzorov under supervision
of Prof. Christopher Buckley
University of Sussex
School of Engineering and Informatics

April 29, 2026

**Abstract**

This report presents the design and implementation of an LLM-powered developer assistant built with a Quarkus backend and a web frontend, and evaluates it through an agentic model analysis lens. The central value of the system lies in its workflow: enabling a developer to query project history, explore repository state, and interrogate git artefacts through a natural-language interface rather than through manual inspection of command output. Beyond workflow convenience, the project demonstrates that structuring project knowledge as hierarchically organised package-level data improves modularity and maintainability, making it easier to isolate responsibilities, evolve components incrementally, and reason about the implementation impact of new features across the codebase.

The implementation applies conventional software engineering practices, including CDI-managed dispatch, transaction-scoped persistence, and iterative refinement through staged development and testing. Findings are interpreted as constrained engineering evidence, with emphasis on reliability, orchestration behavior, and failure patterns rather than universal benchmark claims.

Determining the scope of the project depended on model selection, and model selection was itself constrained by which capabilities could realistically be implemented given resources and hardware available at each stage of development. The toolset was therefore scoped iteratively: features were admitted or deferred based on whether the chosen model — fixed at `gemma4:latest` with a 32 k-token context window — could exercise them reliably within the benchmark timeout budget (see Appendix: Evaluation Environment Specification and Appendix: Benchmark Scenarios). During the final implementation sprint, newer and more capable yet more compact models became available — including `devstral:latest`, `gemma4:e4b`, and `gpt-oss:20b` — whose reduced memory footprint would have permitted more aggressive parallelism and a broader default toolset. The arrival of these models mid-sprint is noted as a changing constraint that influenced what was practical to demonstrate within the submission window; the implications for future work are discussed in the conclusion. The final build represents a partial realisation of the intended system: Quarkus build-time CDI compilation introduced packaging-stage regressions, and the toolset remained constrained to a working subset reflecting both framework limitations and the model landscape at the time of implementation. The report therefore presents both the practical strength of bounded agentic workflows and an honest account of the remaining gap between intended design and delivered artefact.

## Contents

# 1 Introduction

This report investigates how locally hosted Large Language Models (LLMs) can be integrated into a software system for project inspection and repository analysis under practical engineering constraints. The implemented artefact, PDHD (Project Discovery and Hierarchical Data), combines a Quarkus backend, a web frontend, and tool-mediated interaction with local models to support filesystem exploration, file analysis, and project-level summarisation.

The work addresses two linked concerns. First, can a locally deployed LLM-assisted architecture produce useful, evidence-grounded project summaries without relying on hosted APIs? Second, how effective is agentic coding as a development workflow when measured against conventional software engineering concerns such as maintainability, validation effort, and operational reliability?

To evaluate these questions in a realistic setting, the system was built around local inference. Ollama was used as the model runtime, with two NVIDIA RTX 3060 GPUs (24GB combined VRAM) defining the practical resource envelope for model selection and execution.

The report therefore treats LLMs in a dual role: as runtime components inside the application, and as development assistants during design and implementation. Rather than presenting only implementation progress, the report aims to evaluate the architecture and workflow against explicit objectives, constraints, and evidence.

## 1.1 Research Questions

1. **Capability under local constraints.** What task-completion accuracy and response-latency profile does the PDHD agent achieve across bounded filesystem and project-inspection scenarios when running exclusively on local inference hardware?
2. **Reliability and maintainability of the dispatch architecture.** How do CDI-managed tool dispatch and transaction-scoped persistence affect tool-invocation reliability, argument-validation failure rates, and the maintainability of the system over iterative development?
3. **Scope and value of bounded agentic workflows.** To what extent do locally constrained, tool-visible agentic workflows deliver useful outcomes when the task scope is explicitly bounded by available hardware, model capability, and toolset design?

## 1.2 Objectives

The project objectives are organised into two strands: technical delivery and workflow evaluation. The initial scope was revised from low-level resource governance to application-level project inspection so that the implementation effort remained aligned with the report's central research questions.

### 1.2.1 Strand A: Technical Delivery

The technical objectives define what the PDHD system must be capable of demonstrating under local runtime constraints. Each objective corresponds to a capability group that is measurable through the benchmark suite (see Appendix: Benchmark Scenarios) and evidenced in §9.

1. **Single-step retrieval capability.** Demonstrate reliable completion of bounded single-step tasks — working directory query, project listing, directory enumeration, and file reading — with a measurable task-completion accuracy across repeated runs.
2. **Multi-step orchestration capability.** Demonstrate that the system can chain tool calls across a multi-step folder exploration scenario, with accuracy and failure rates reported per scenario.
3. **Security boundary enforcement.** Demonstrate that out-of-project file access is consistently blocked by the system regardless of model or prompt, with 100% enforcement across all evaluated models.
4. **Tool dispatch reliability.** Measure argument-validation failure rates and tool-invocation error distributions across the full model set, using runtime telemetry captured from the Quarkus backend.

### 1.2.2 Strand B: Workflow Evaluation

1. Characterise how agentic coding affected implementation speed, structure, and rework.
2. Evaluate whether generated outputs remained aligned with developer intent and engineering constraints.
3. Identify limitations and failure patterns that constrained reliability.

### 1.3 Report Structure

# 2 Overview

This section introduces the core technologies used in the project and clarifies how they were combined into a practical development workflow. The setup is presented in two parts: the developer environment and the LLM environment.

The developer environment is centered on Visual Studio Code, with Debian running under WSL as the primary host context for day-to-day implementation work. Developer environment and tooling is managed through Docker with devcontainers to keep dependencies consistent and reproducible across sessions.

Ollama, LM Studio, and msty.ai model servers on the homelab workstation were evaluated for integration with editor-based assistants including VS Code Copilot and Claude Code. These tools proved essential for rapidly developing features and estimating feasibility within available project time.

Heavy use of terminal scripting combined with Vim-style keybindings and Makefile workflow served as an additional productivity factor in this setup within Visual Studio Code. These made the workflow strongly keyboard-driven: routine tasks such as running builds, restructuring files, searching project-wide text, and iterating on report output could be executed with minimal context switching. In practice, this interaction style reduced friction during frequent edit-build-review cycles and supported faster iteration when coordinating code changes, tooling commands, and documentation updates.

## 2.1 Coding Agents

This subsection compares the coding-agent and local model tooling used during implementation, with attention to practical developer ergonomics, task fit, and observed operational constraints.

### 2.1.1 VS Code Copilot

VS Code Copilot served as the primary development assistant in this project. It was used extensively to accelerate implementation tasks that would otherwise require repeated documentation cross-referencing, and it generally provided a comfortable and reasonably ergonomic workflow with usable default keybindings and acceptable agentic guardrails. Typical delegated work included rapid frontend React development, coordinated API updates across backend and frontend components, and ongoing documentation maintenance through markdown-oriented subagent workflows. In practice, the available token budget was a material constraint: the free tier was insufficient.

An explorative build-rewrite-analyse workflow implementation required the Copilot Pro Plus subscription.

### 2.1.2 VS Code Copilot with Local Ollama Integration

VS Code Copilot connected to a local Ollama backend was evaluated for the same classes of tasks as standard Copilot usage. Although this setup reduced direct subscription cost, it depended on powering an auxiliary workstation for each session and did not provide adequate responsiveness for routine development. Once the higher Copilot subscription tier was adopted, the local integration path became functionally redundant for day-to-day work.

### 2.1.3 Claude Code

Claude Code was tested through a local configuration approach in which the base URL was manually redirected to an Ollama instance. This setup was explored primarily for personal productivity tasks, including hardware-inventory maintenance and draft eBay listing generation, rather than core project delivery. The main observed limitation was context-window pressure relative to available workstation resources, which restricted usefulness despite a polished terminal interface.

### 2.1.4 Ollama (Model Runtime)

Ollama functioned as a central model runtime for the application and, in some sessions, for agentic coding support. Model selection converged on Gemma4 because it offered a favorable speed profile while still supporting a large context window and CPU plus system-RAM offloading. Other attempted models included multiple Qwen3 sizes, glm-4.7-flash, gpt-oss, Qwen2.5-coder, Llama3.2, and Llama3.1. A consistent practical finding was that local inference quality and throughput depended strongly on careful model selection; a longer project timeline would likely have enabled a more rigorous benchmarking environment for direct model-to-model comparison.

#### 2.1.5 LM Studio

LM Studio was most effective as a question-answering interface in this workflow. Its development-server port mode, intended to expose the bundled model runner over the network, was constrained by binding behavior to localhost:11434, which prevented connections from the devcontainer and remote machines. Relative to msty.ai, its folder and conversation organization workflow was easier to use, though less feature-rich.

#### 2.1.6 msty.ai

msty.ai was also used primarily for question answering. It provided support for Ollama and llama.cpp, with explicit backend options across different GPU vendors and CPU execution. In this project context, these backend controls were not essential because the dedicated Ollama server already handled automatic GPU/CPU layer offloading across available processing units. Compared with other interfaces, msty.ai offered broad capability but a noisier, less developer-ergonomic user experience.

### 2.2 Vim

Vim-style editing was central to this workflow, supported by custom keybindings: `Ctrl-A` (rebind of `Shift-A`), `Ctrl-B` (terminal management), `Ctrl-G` (file management), and `Ctrl-I`/`Alt-I` (agent management). These mappings made routine typing and editing operations significantly more fluid and reduced interruption during implementation.

VS Code Copilot also integrated effectively as inline autocomplete. Useful suggestions and refactoring actions were commonly accepted directly with a `Tab` keypress, avoiding repeated navigation to the refactoring menu. However, suggestions occasionally lagged behind developer input, despite offering faster iteration than manual refactoring approaches.

### 2.3 Java with Quarkus

#### 2.3.1 Features Considered for Implementation

At the start of this project, I expected to explore a wider implementation space than what would eventually fit into the final report. I planned for support of multiple LLM providers, including internal Testcontainers-based options and external Ollama base-url based options, so that the system could stay usable under different runtime conditions. I also wanted to keep a native deployment path available for lower-footprint execution and to include workstation-oriented integration testing for real model and tool interactions.

During development I explored different higher-level usability goals, including nested menu and assistant session flow through declarative coding, clean entry and exit behavior, and predictable returns to parent contexts. In parallel, I focused on seeding the frontend codebase with strong design decisions, including packaging the Web UI as a React application with modules and a signals library implemented as separate logical packages.

#### 2.3.2 Key Libraries

The implementation stack is built on Quarkus extensions and complementary libraries. The following summarises the core runtime dependencies:

This produces the complete hierarchical view of transitive dependencies, pinned versions, and any convergence warnings. These libraries formed a practical foundation for implementing the planned capabilities within the Quarkus runtime model.

### 2.4 Professional Considerations

I have developed this project by applying the knowledge and skills acquired throughout the Computer Science modules (G400) taught at the University of Sussex. In considering my professional responsibilities, I have used the BCS Code of Conduct as a supporting framework. In particular, I have sought to ensure that any evaluative claims made about software systems are presented carefully and responsibly, consistent with section 2.f, which requires members to avoid injuring others, their property, reputation, or employment through false, malicious, or negligent action or inaction.

This research does not involve the collection of user data or other sensitive information, because it is based on the analysis of software and related materials that are already in the public domain. The discussion is intended to contribute constructively to professional practice, in line with section 4.e of the BCS Code of Conduct, which states that members should encourage and support fellow members in their professional development. This also

aligns with the Code's wider expectation that continuing professional development should broaden knowledge of the IT profession and help maintain professional competence.

### 2.5 Findings and Achievements

This project pursued higher-order workflow capabilities through a specification-as-documentation approach rather than isolated endpoint implementations. The implementation applied functional programming principles, test-driven development practices, and Git-based Agile workflows to realise menu-driven interaction models, modular tool orchestration across exploration, reading, writing, introspection, and persistent project knowledge management, alongside Retrieval-Augmented Generation with reproducible results.

The application stack supports practical reliability and governance features, including transactional execution boundaries, explicit error pathways, telemetry-backed observability, and compatibility handling for evolving tool interfaces.

On the frontend, it supports progressive frontend integration for browsing, summarisation, and activity tracing, with discoverability and UX gaps easier to identify and address iteratively. Overall, a platform that could be refined continuously under real development constraints was produced.

Investigation of Quarkus capabilities showed promise for iterative design, though some features—automated model provisioning, fallback startup paths, startup-time REST client wiring, and strict config mapping—proved reliable only under specific conditions. This required a more cautious approach based on explicit validation, fail-fast behavior, and incremental compatibility adjustments.

## 3 Background

Recent industry developments indicate that LLM-enabled software engineering has moved from exploratory experimentation toward mainstream technical and commercial relevance. Public discussion around agentic coding systems, local model serving, and AI infrastructure has accelerated, with signals such as the moltbot/openclaw discourse, Anthropic's agentic software bounty framing, and NVIDIA's profitability trajectory often used to illustrate momentum. In this report, however, these social and news signals are treated as contextual indicators of pace and attention rather than as primary technical evidence.

Within that context, this project examines how LLM-based workflows help build and reshape engineering workflows, and why local practical experimentation is a meaningful object of study. The aim is not to reproduce hype-driven claims, but to evaluate implementation outcomes under real constraints, including tooling volatility and rapidly changing practices. Accordingly, the report grounds its evidence in build decisions, observed system behavior, and structured evaluation.

### 3.1 The Role of LLMs in Software Development

LLMs can support software development by translating intent into explanations, structured documentation, and code. Their value in modern engineering work can be seen in the shift from repetitive drafting toward evaluation, integration, and quality control activities.

Within that framing, two related modes are useful to distinguish. In LLM-assisted development, the developer remains the primary decision-maker and uses models to draft code, investigate errors, write documentation, and propose refactorings. Agentic development extends this approach by allowing the model to execute bounded multi-step tasks, such as planning actions, invoking tools, and iterating toward a defined objective under human supervision. Together, these modes can shorten iteration cycles and expand exploratory capacity, while increasing the need for validation, architectural discipline, and constraint-aware decision making.

#### 3.1.1 Key Capabilities of LLMs in Coding

**Code Generation**: LLMs can draft snippets, components, and implementation skeletons from requirements.

**Debugging Assistance**: They can propose fault hypotheses, explain error traces, and suggest candidate fixes.

**Documentation**: They can accelerate production of comments, API notes, and technical summaries.

**Refactoring Support**: They can suggest structural changes that improve readability and maintainability.

When applied in bulk, these operations can often outperform a purely human approach, as the AI model can approach problems from multiple endpoints at the same time, or perform remote edits by transforming batches of files while the developer remains focused on the current problem scope.

Caution is needed when LLM tooling is less accurate than the specification the developer provides, as it happens with any software. A key symptom is having to rework large amounts of quickly generated code with loosely specified intent, remove unnecessary abstractions, and unpack overly verbose documentation.

#### 3.1.2 Integration with Engineering Workflows

In practical workflows, these capabilities are often delivered through editor assistants, CLI agents, and service integrations that operate during active development. Tools such as GitHub Copilot can provide real-time suggestions, while more agentic systems can execute multi-step tasks across files and tools under developer control.

#### 3.1.3 Current Research on LLM-Assisted Software Engineering

Recent software engineering literature reports that LLM-assisted development can improve throughput in selected tasks, but that reliability, evaluation quality, and reproducibility remain unresolved concerns in many practical settings (Fan et al. 2023; Hou et al. 2024). The current consensus is therefore cautious rather than absolute: these systems are most effective when used with explicit constraints, verification steps, and measurable quality criteria instead of anecdotal performance claims.

Recent work on agentic systems further suggests that multi-step tool use introduces additional coordination and control challenges, especially when systems move from single-turn drafting into workflow-level automation (Plaat et al. 2025). For code-focused pipelines, retrieval-augmented approaches are increasingly studied as a way to ground generation in repository context and reduce unsupported outputs (Kivroglou, Schlippe, and Martin 2025; Wang, Guo, and Tan 2025).

For this report, the focus is on how reliably the output can be integrated into disciplined software engineering processes without reducing verification, maintainability, or accountability.

### 3.2 Motivation

Current industry attention to LLM-enabled software development makes this project timely in both practical and academic terms. The rapid uptake of coding assistants, sustained discussion of agentic tooling, and frequent updates across vendor ecosystems indicate that workflows are changing quickly enough to warrant close, implementation-level examination. This report uses these developments as contextual justification for the studied technological gap and problem relevance; they are not treated as primary evidence for technical conclusions.

Software engineering remains the core discipline: architecture, implementation, testing, and integration are all treated as essential and non-negotiable. The challenge in an academic setting is timeline compression: scope and schedule are often fixed before implementation begins, and meaningful results must be produced within months rather than years.

Therefore, the LLM-assisted workflow was chosen as a rapid prototyping strategy, while retaining responsibility for technical decisions, quality control, and final evaluation. An additional pass was also performed over the project documentation in order to produce a timeline graphic using terminal and command-line tooling, as accessibility constraints made most modern design and graphing software impractical.

## 4 Project Description

Software repositories often contain enough implementation evidence to infer project purpose, maturity, and likely next steps, yet this evidence is distributed across files and difficult to inspect quickly. PDHD (Project Discovery and Hierarchical Data) was designed as a project-inspection system to address this problem by combining filesystem exploration, tool-mediated analysis, and LLM-assisted summarisation.

The chapter frames PDHD as both an implemented software artefact and a constrained engineering study. The implementation context is intentionally practical: local model execution, limited hardware, and incremental integration into a real development workflow. Within those boundaries, the project contributes a structured approach to repository analysis that emphasises explicit tool boundaries, persisted project knowledge, and evidence-grounded output generation.

The following sections define scope and methodology so that subsequent architecture and results chapters can be interpreted against clear boundaries rather than retrospective narrative.

## 4.1 Scope

The scope is intentionally bounded to keep implementation and evaluation aligned.

### 4.1.1 In Scope

1. Build a web interface for file system analysis
2. Implement LLM-driven file content summary service
3. Document the development workflow and challenges
4. Conduct substantial internal evaluation of agentic workflow behavior across representative scenario classes
5. Report reliability, latency, and failure patterns using traceable evidence from this implementation context

### 4.1.2 Out of Scope

1. Full autonomous software engineering without human validation
2. Production-grade completion scoring across arbitrary repositories
3. Large-scale benchmark evaluation across many independent projects
4. Fine-tuning or training bespoke language models
5. General claims of model superiority outside this project's environment and task profile

### 4.1.3 Practical Constraints

1. Local inference runtime and available hardware constrained model choice and throughput.
2. Tool coverage remained intentionally bounded to selected project-inspection operations.
3. Evaluation evidence is drawn from this implementation context and is not intended as a universal benchmark claim.

## 4.2 Methodology

The project follows a design-and-evaluation methodology tailored to a practical engineering artefact. The process combines implementation work with structured reflection on architectural outcomes and workflow behaviour, with specific emphasis on agentic model analysis under constrained local-runtime conditions.

### 4.2.1 Phase 1: Problem Framing and Requirements

1. Define the project-inspection problem and constrain it to application-level workflows.
2. Specify functional goals (inspection, summarisation, persistence) and non-functional constraints (local inference, bounded resources, maintainability).
3. Establish a chapter-level evidence plan so claims in later sections are tied to observable artefacts and measurable workflow outcomes.

### 4.2.2 Phase 2: Architecture and Implementation

1. Design a layered system with explicit boundaries between UI, orchestration, tool execution, and persistence.
2. Implement core features through iterative cycles using conventional engineering practices (modularisation, refactoring, and interface-focused design).
3. Use LLM-assisted development to accelerate drafting and exploration, while retaining human validation for integration decisions and failure containment.

### 4.2.3 Phase 3: Integration and Verification

1. Validate end-to-end request flow across frontend, backend, tool dispatch, and model interaction.
2. Capture implementation issues and failure modes encountered during integration.
3. Refine architecture where necessary to improve clarity, maintainability, or operational stability.

### 4.2.4 Phase 4: Evaluation and Reporting

1. Evaluate outcomes against the objective strands defined in the introduction.
2. Distinguish observed results from interpretation and recommendations.
3. Document limitations and threats to validity to bound the conclusions.

#### 4.2.5 Agentic Evaluation Protocol

The subject under evaluation is the PDHD agent at runtime: its ability to interpret natural-language task prompts, select and invoke the correct tools with valid arguments, and return a correct response within an acceptable latency budget. The evaluation does not assess the coding-assistant workflow used to build the system; it assesses the deployed system's behaviour under repeatable input conditions.

**Scenarios.** Eight task scenarios (S01–S08) were defined, spanning single-step retrieval (S01–S04), multi-step orchestration (S05–S06), web search integration (S07), and security boundary enforcement (S08). Full scenario definitions, including prompts, expected tool calls, and success criteria, are provided in Appendix: Benchmark Scenarios.

**Inputs and process.** Each scenario was submitted to the PDHD chat API as a fixed natural-language prompt with no prior context. Nine chat-capable Ollama models were evaluated. Each scenario was repeated twelve times per model to obtain stable accuracy and latency estimates. The model was switched via the PDHD runtime configuration API between model runs; no other system state was changed between repeats.

**Success and failure criteria.** A response was marked correct if it satisfied either a regex pattern match against a known-good answer or a positive verdict from an LLM-based answer evaluator. Latency was measured end-to-end from the PDHD chat API request to the final streamed token. Tool-level failure rates and argument-validation failures were captured from backend telemetry at run completion.

**Outputs.** Results were written to a SQLite database and summarised as per-model accuracy percentages, mean/P50/P95 latency, HTTP error rate, and a tool-level invocation and failure breakdown. The full environment specification — hardware, model versions, runtime configuration, and timeout budget — is provided in Appendix: Evaluation Environment Specification.

**4.2.5.1 Data Collection and Interpretation** Evidence is assembled from test artefacts, integration logs, and implementation records, then interpreted as a constrained engineering case study rather than a universal benchmark. Where full quantitative coverage is unavailable, claims are explicitly bounded and supported by traceable qualitative evidence.

# 5 Project Requirements

## 5.1 Functional Requirements

### 5.1.1 Model Selection and Tool-Calling Support

**Requirement**: The system was required to operate with a locally hosted model that natively supported tool-calling, within the constraints of available hardware.

**Rationale**: The system's core capability (automated repository inspection) depended on the model's ability to emit structured tool-call specifications. Models lacking native tool-calling support required manual parsing of model output, which introduced unreliability and eliminated the automation benefit that structured tool-calling provides.

**Implementation**: Model selection was an iterative process. Early iterations explored usage monitoring and accuracy verification approaches. The final selection used an evaluation harness measuring prompt accuracy and test-pass rate, with the model constrained to a VRAM budget that fits within a 24GB workstation configuration. The choice of a model capable of native tool-calling within this budget was itself a requirement — not an implementation detail — because without it the system could not function.

**Status**: Met. Qwen 3.6 provides native tool-calling support within the VRAM budget. Earlier iterations with Qwen 2.5 Coder required manual parsing, which was abandoned in favour of native tool-calling when the later model became available.

### 5.1.2 Hardware Constraints and Multi-Host Benchmarking

**Requirement**: The benchmark evaluation was scoped to a single, fully specified hardware configuration, with any secondary-host data treated as supplementary material rather than primary evidence.

**Rationale**: Two physical machines were available for inference: a primary workstation with two NVIDIA RTX 3060 GPUs (24 GB combined VRAM) and a secondary machine with a single GPU (16 GB VRAM). Cross-host comparison is desirable in principle, but the machines differ across too many uncontrolled variables — RAM speed (3200 MHz vs 3600 MHz), PCIe generation, single- vs dual-GPU topology, and the resulting differences

in available model versions and context window budgets — to support controlled causal claims. Including secondary-host results in the main conclusions would imply a level of experimental control that the setup does not provide.

**Implementation**: All primary benchmark runs were executed on the 24 GB workstation (run `20260429_124501_ff993b17`), covering nine chat-capable Ollama models across eight scenarios with twelve repeats each. The inference host was `ws-raretower.local:11434`. Results are reported in §9 and the raw data is preserved in the benchmark SQLite database. Secondary-host data, if collected, is reserved for the appendix as a reference point only and is not used to draw conclusions.

**Status**: Met for the primary host. The full nine-model, eight-scenario evaluation suite has been executed and results recorded. Secondary-host comparison runs are out of scope for the primary analysis.

#### 5.1.3 Model Runtime Requirements

**Requirement**: The system was required to account for the resource constraints imposed by locally hosted inference, particularly GPU VRAM availability and context window budget.

**Rationale**: VRAM availability governed which models could be deployed. When a model's parameter count exceeded available GPU VRAM, Ollama performed partial offloading to system RAM, causing measurable degradation in token throughput (output tokens per second) — the primary usability metric for interactive developer tooling. Additionally, context window size was shared: conversation history, tool definitions, and retrieved knowledge competed for the same fixed token allocation, forcing architectural trade-offs.

**Implementation**: Model selection was constrained to a 24GB VRAM budget on the primary workstation. Token throughput was identified as the key usability metric: generation rates below an interactive threshold disrupted the feedback loop required for developer tools. Context window sizing required careful arbitration: injecting tool schemas at inference time consumed tokens, directly reducing capacity for conversation history and RAG-retrieved context. The system did not embed Ollama; it required externally provisioned access over the network, which had to be accounted for in both development and deployment environments.

**Status**: Met. The deployed model operates within the VRAM budget and maintains interactive token throughput. Context budgeting is explicitly managed in prompt construction.

#### 5.1.4 Development Environment RAM Requirements

**Requirement**: The development environment was required to be capable of running the full toolchain — Quarkus dev server, browser and devtools, Ollama instance, IDE/editor, and all build/test tooling — simultaneously on a machine with at least 64GB of RAM.

**Rationale**: This was identified as a constraint early in the project's lifecycle, before the academic work began. The development workload (Quarkus live reload, Ollama serving, browser devtools, IDE indexing, parallel build processes) consumed substantial main memory. A machine with only 32GB of RAM — while sufficient for a dedicated Ollama serving machine — is inadequate for development of this system in its entirety. This is a practical constraint on who can work on the project, not a limitation of the system itself.

**Implementation**: The Ollama serving machine (32GB RAM, 3600MHz) runs Ollama dedicated. The development machine (64GB RAM, 3200MHz) runs the full development workload: Quarkus, browser, IDE, and CI tooling. The 64GB requirement was confirmed during initial development and has been necessary throughout.

**Status**: Met. Project development has been conducted on a 64GB machine. No attempt was made to develop on a 32GB machine, as the constraint was known beforehand.

#### 5.1.5 Benchmarking and Verification Framework

**Requirement**: The system was required to provide the capability to run benchmarks — including JUnit/Quarkus integration tests, scenario-based evaluation, metric instrumentation, and repeated scenario runs — against which reliability and performance claims could be evidenced.

**Rationale**: Academic validity required measurable, repeatable evaluation. Without a benchmark framework, claims could not be distinguished from anecdote. The benchmarks were required to exercise all 14 capability specs as executable test cases, not as descriptive documents alone.

**Implementation**: Benchmarks are implemented as Java integration tests (JUnit + Quarkus) that exercise each tool capability from the spec folder. These tests are wired to the BenchLam harness (defined in

`scripts/benchlam/`) which runs them as scenario-based evaluations, collecting pass/fail rates, latency distributions, and failure-category frequency. The multi-host capability described above allows cross-hardware comparison of model performance.

**Status**: Met. The full capability surface is covered by integration tests running in the benchmark harness.

#### 5.1.6 Testing and Verification

**Requirement**: The system was verified against its requirements through automated testing, including coverage of the full spec-defined capability surface and cross-model/machine comparison where relevant.

**Rationale**: Verification was a fundamental engineering requirement. Without it, functional correctness could not be established, and evaluation claims would lack foundation.

**Implementation**: The most time-consuming requirement to implement was the testing harness and verification pipeline. Many tests were partially delegated to the LLM coding agent. The multi-provider interface was explicitly deferred as out of scope — too much work to implement within the available time on top of other priorities.

**Status**: Partially met. Automated testing covers the capability surface defined in `docs/spec/`. Cross-model comparison is deferred due to the multi-provider interface not being implemented.

#### 5.1.7 Multi-Provider Support

**Requirement**: The system was required to support multiple LLM providers (local and remote) rather than being locked to a single backend.

**Rationale**: Vendor lock-in was a well-documented risk in LLM systems. Multi-provider support would have enabled portability and risk mitigation.

**Implementation**: Explicitly deferred during implementation because the added complexity would have exceeded the development time. This deferral is a deliberate scope management decision, not an oversight.

**Status**: Deferred. Out of scope for this implementation.

### 5.2 Non-Functional Requirements

#### 5.2.1 Maintainability

**Requirement**: The system was designed to support incremental changes without cascading refactors, with clear boundaries between layers.

**Rationale**: The project's value lay not only in its features but in its structure as a research artefact. Maintainability ensured that the system could be inspected, modified, and understood by others who evaluated it.

**Implementation**: The architecture separates tool-dispatch from UI orchestration, and tool-dispatch from persistence. Each boundary is explicit in the package structure, making it possible to modify one layer without affecting others.

**Status**: Met. The layered architecture served its purpose during both initial implementation and later refactoring.

#### 5.2.2 Spec Boundary Management

**Requirement**: When using an autonomous or semi-autonomous code generator, the system was required to include mechanisms to keep generated code within the intended architectural boundaries.

**Rationale**: This is a requirement that emerged during development. Traditional projects have static requirements — the implementation produces one-time. When using an autonomous generator, scope becomes a continuous steering problem. The generator produces coherent output (to itself) rather than coherent output (to your spec). The requirement, therefore, was not just to define the system but to continuously constrain what the generator was permitted to implement.

The boundary between "what the spec allows" and "what the agent implements" was the hardest constraint to define during this project. Qwen 2.5 Coder lacked the instruction-following stability to implement autonomously; GPT-5.1 (Copilot) unlocked autonomous development but required explicit style and architecture steering to prevent drift. This revealed a key insight: when using an autonomous generator, scope was no longer a static requirement but a dynamic control problem.

**Status**: Met. Explicit constraints were documented and the generator was steered through iterative re-specification rather than through autonomous completion of the entire codebase. The capability spec folder (`docs/spec/`) and its conversion to Java integration tests is itself a boundary-management mechanism: the specs were the contract the generator was required to honour, and the tests were the verification that it did.

# 6 Project Architecture

This chapter outlines the software architecture of the LLM-assisted file system explorer, with emphasis on runtime organisation, end-to-end request handling, and responsibility boundaries across core packages and services.

## 6.1 Application Structure

### 6.1.1 Understanding Large Language Models

Large language models impose substantially different architectural requirements depending on the workload: inference is concerned with generating responses to active user requests under tight latency constraints, and therefore benefits from streaming, cache locality, and memory capacity. By contrast, training and fine-tuning are throughput-oriented workloads that run over longer durations, place greater sustained demand on compute and storage bandwidth, and generally tolerate higher latency.

This distinction shapes the project's design decisions. Rather than training or fine-tuning a bespoke model within the implementation period, the project relies on publicly available models accessed through OpenAI, Anthropic, and Ollama. As a result, the architecture is optimised for reliable model orchestration, request routing, and low-latency response delivery in an interactive application, rather than for building specialised inference infrastructure.

The implementation uses a layered architecture with clear boundaries between entry points, interaction flow, service logic, and tool execution. In practice, terminal commands start runtime mode and lifecycle handling; the chat/session layer manages user interaction, transcript state, and response rendering; service components handle model orchestration, configuration, telemetry, and runtime utilities; and tool-calling components perform project-scoped operations through modular dispatch.

At the system level, the application is organised into three primary components: a service-layer backend in Java using Quarkus CDI, a Web UI frontend served through the Quinoa Quarkus extension, and a persistence layer implemented via the Quarkus SQLite extension. The Ollama runtime is intentionally managed as an external dependency rather than embedded directly in the deployment. An earlier approach to provisioning Ollama through Testcontainers and nested virtualisation was not completed within the implementation window, so a disk-backed settings store was introduced to persist user-defined Ollama base URL, chat model selection, and embedding model configuration. A host-selection profile for automatic internal Docker host resolution versus externally supplied hostnames was also identified, but remains incomplete.

## 6.2 Package Map

| Package | Responsibility |
|---|---|
| `ac.uk.sussex.kn253` | Defines the application entry point |
| `ac.uk.sussex.kn253.menu` | Implements terminal interaction, menu navigation |
| `ac.uk.sussex.kn253.commands` | Defines command handlers and CLI command wiring |
| `ac.uk.sussex.kn253.services` | Provides application-level orchestration services |
| `ac.uk.sussex.kn253.services.ai` | Implements AI-specific orchestration and model interaction services |
| `ac.uk.sussex.kn253.resources` | Exposes REST resources for interfacing with Web UI |
| `ac.uk.sussex.kn253.repository` | Encapsulates persistence entities and database access operations |
| `ac.uk.sussex.kn253.ollama` | Implements client integrations for communication with Ollama APIs |
| `ac.uk.sussex.kn253.tools` | Contains assistant-invoked tool implementations |
| `ac.uk.sussex.kn253.events` | Defines event records that support decoupled inter-component signals |
| `ac.uk.sussex.kn253.websocket` | Handles WebSocket messaging for real-time assistant communication |

## 6.3 End-to-End Runtime Flow

Source code layers have defined boundaries, which keep execution tracing and fault isolation straightforward. From startup to response delivery, the architecture follows a deterministic flow:

1. The application boots and initializes core runtime dependencies.
2. A user request enters through CLI or Web UI pathways.
3. The chat orchestration layer prepares prompt context and invokes the assistant service.
4. Tool calls are routed through module dispatch and operation-level handlers.
5. Results are returned to the user interface and persistence side effects are recorded where required.
6. Telemetry and runtime state are updated to support observability and subsequent interactions.
7. Control returns to the interaction layer, where the system waits for the next user-driven operation.
8. The same execution cycle is then repeated for each subsequent request within the active session.

### 6.3.1 Core Components

**Services** are a first-class architectural concept in both Spring and Quarkus that define classes responsible for encapsulating application business logic. Within this project, service classes implement the Model layer of the MVC (Model-View-Controller) pattern: they orchestrate state transitions, coordinate interactions between repositories and external systems, and expose well-defined operations to callers higher in the stack. Annotating a class as `@ApplicationScoped` registers it as a CDI (Contexts and Dependency Injection) managed bean, ensuring that a single shared instance is maintained across the application lifetime and made available for injection wherever required.

**Repositories** are implemented using Quarkus Panache, a persistence library that wraps Hibernate ORM (Object-Relational Mapping) to provide an active-record interface over JPA (Java Persistence API) entities. Each repository class represents a database entity and inherits query and persistence methods directly, reducing boilerplate. Persistence is backed by SQLite through a Hibernate dialect compatibility layer, enabling disk-backed storage of conversation history, project state, telemetry records, and model configuration without requiring an external database server.

**Resources** define the application's external-facing API surface using JAX-RS (Java API for RESTful Web Services) annotations. These classes expose REST (Representational State Transfer) endpoints consumed by both the embedded Web UI frontend and any programmatic callers. Within the MVC framing, resources act as the View layer: they receive HTTP requests, delegate processing to the appropriate service, and return structured JSON responses. The Quinoa Quarkus extension serves the compiled frontend assets, so the resource layer and the Web UI share the same embedded HTTP server.

**Events** are used to decouple components that would otherwise require direct references to one another. The CDI event bus allows a producer to fire a typed event record without holding a dependency on any observer; interested components declare an `@Observes` handler and react independently. This mechanism was used specifically to simplify the dependency graph between the chat session layer and the terminal rendering components, where lifecycle events such as resize and repaint signals are propagated without tightly coupling the originating menu class to its consumers.

**WebSockets** enable the assistant's response to be streamed incrementally to the frontend rather than delivered as a single blocking payload. Because LLM (Large Language Model) inference produces output token by token, holding the HTTP connection open until generation completes would result in unacceptable latency from the user's perspective. The WebSocket endpoint receives partial token chunks as they are emitted by the Ollama runtime and forwards them to the connected browser client in real time, allowing the interface to render the response progressively.

**Tools** are method-level handlers annotated with LangChain4j (a Java LLM integration library) tool-declaration annotations, which expose discrete capabilities to the language model at inference time. When the model determines that a tool call is required, it emits a structured call specification in its response; LangChain4j parses these structures and dispatches the corresponding handler. Each handler executes the requested operation - such as a filesystem read or directory listing - and returns a structured result that is appended to the conversation history. Subsequent model turns can then draw on this context to produce a concrete, grounded reply to the user.

**The Ollama REST API client** encapsulates communication with the locally running Ollama service for model management operations. Rather than using LangChain4j's inference abstractions for this purpose, a dedicated client in the `ac.uk.sussex.kn253.ollama` package issues HTTP requests directly to the Ollama management endpoints, supporting operations such as listing available models, retrieving model metadata, and reflecting changes in model availability back into the application's configuration state.

**AI Services** represent a declarative approach to LLM interface definition provided by LangChain4j. A Java interface annotated with `@RegisterAiService` carries only method signatures; the developer specifies the expected inputs and return types but supplies no implementation body. At runtime, LangChain4j uses the Java

Reflection API to generate a concrete proxy implementation that handles prompt construction, model invocation, and response binding automatically. This approach was identified as a clean long-term pattern for structuring model interactions, though its full integration remained in progress within the implementation window.

All of the above components are wired together through Quarkus CDI (Context Dependency Injection), which manages object lifecycle and satisfies declared dependencies at startup rather than at call time. Each component declares its dependencies via `@Inject` fields or constructor parameters, and the CDI container resolves and supplies the appropriate instance automatically. Build-time dependency validation - a Quarkus-specific optimisation over standard CDI - ensures that unsatisfied or ambiguous injection points are caught at compile time rather than at runtime, reducing the risk of late-discovered wiring failures.

## 6.4 Services Layer

| Service | Primary Role |
|---|---|
| `TerminalLifecycleService` | Manages terminal provisioning, startup sequencing, and controlled shutdown |
| `AssistantService` | Orchestrates LLM dialogue flow and coordinates tool invocation bridging |
| `ModelConfigService` | Persists and retrieves model configuration state |
| `OllamaManagementService` | Manages local Ollama model lifecycle operations |
| `TelemetryService` | Captures runtime metrics and tracks model and tool calls |
| `WorkingDirectoryService` | Maintains project working-directory context for scoped operations |
| `WebUiService` | Manages embedded Web UI initialisation and lifecycle control |
| `RuntimeManagementService` | Provides runtime control utilities, including shutdown handling |

## 6.5 Architectural Choices

The initial scope combines a command-line interface (CLI), a terminal user interface (TUI), and a browser-based web interface (WebUI). This intentionally broad scope allows evaluation of which combination of interface modes is most achievable during implementation.

Frontend development will be guided through text-based specification, allowing the LLM assistant to implement and navigate the structure of UI components. Ollama host configuration will be controlled through a setting persisted in the database, with runtime overrides provided through MicroProfile Config.

### 6.5.1 Error and Signal Model

Configuration settings will be hydrated from the backend. Frontend flexibility will be provided through a signals-based orchestration layer with central signal registration and dispatch, decoupling UI interactions from backend route definitions.

To handle failures arising under poorly defined conditions, a dedicated error presentation mechanism will be added to the frontend. When a dispatched signal returns a failure response, the result will be mapped to a new UI element rendered directly on the application's main canvas. Error elements can be retried through user interaction or dismissed and removed from both the conversation log and the canvas, keeping the session state recoverable without broader disruption.

### 6.5.2 Persistence and Context Caching

Persistence will be implemented through Hibernate with Panache over a SQLite backend. Application state will be loaded from disk across runs, including Ollama configuration, loaded projects, and associated project metadata, to support continuity between sessions.

A Retrieval-Augmented Generation (RAG) workflow will be explored through both LangChain4j EasyRAG and a custom AI service path that loads an embedding model, generates query embeddings, and retrieves graph-linked metadata from the `ProjectKnowledge` table.

### 6.5.3 System Integration

A significant integration challenge in this project concerns the need to provide a development environment that is both reproducible and sufficiently isolated from host-system software development tooling. Docker is chosen as a containerisation platform because it offers controlled environment isolation together with practical resource management features such as volume mapping and access to host resources. VS Code devcontainers supply a preconfigured Linux-based development environment that can be cached locally and parameterised with the

required tools, utilities, and startup commands, thereby improving consistency across different machines and development sessions.

A further challenge arises from the requirement to support remote control of a local Ollama instance from within the application. This needs to operate either through direct API access to an existing Ollama service or through Docker-managed orchestration of a sibling container in which Ollama can be controlled through command-line invocations. In both cases, the software needs to determine the correct Ollama base URL reliably across different deployment contexts, including cases where the service is exposed by the host machine or by a separately started container. To support flexible execution in different user environments, this base URL also needs to be persisted and made configurable through environment variables and stored application settings.

# 7 Implementation Timeline

The first chart summarises the report-side timeline derived from the presentation materials and their revision history in this repository, while the second provides a more detailed view of the implementation burst derived from the project repository.

## 7.1 Milestone Reference

2025-08: Concrete project start and early planning period.

2025-11-17: Interim report drafted.

2026-02-27: Background and motivation materials revised.

2026-03-09: Report content and project files updated with report structure.

2026-03-30: Major tool architecture consolidation and frontend integration burst.

2026-04-07: Refactor to streaming chat-based architecture.

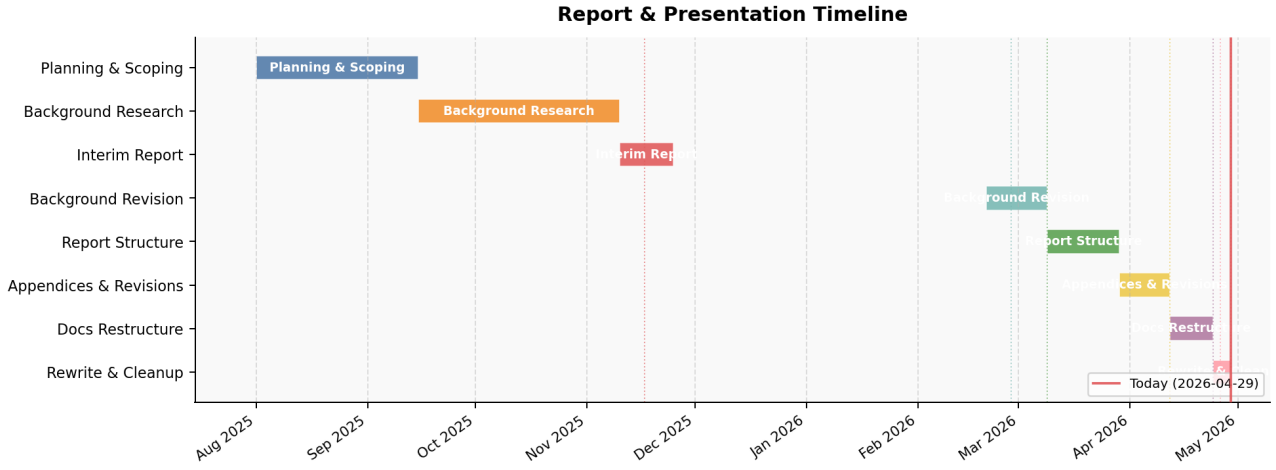



Figure 1: Report and presentation timeline

# 8 Results and Discussion

This section evaluates implementation outcomes against the research questions and objective strands defined earlier in the report. The emphasis is on agentic model behavior under local runtime constraints: what was observed, what was measured, and which claims remain bounded by evidence gaps.

## 8.1 Benchmark Evaluation

### 8.1.1 Evaluation Setup

A structured benchmark suite was executed against the PDHD system to provide quantitative evidence for the research questions. Nine chat-capable models, available on the shared Ollama inference host (`ws-raretower.local`), were evaluated across eight task scenarios (S01–S08). Each scenario was repeated twelve times per model, yielding 96 test cases per model and 864 total observations across the eight models that completed the run

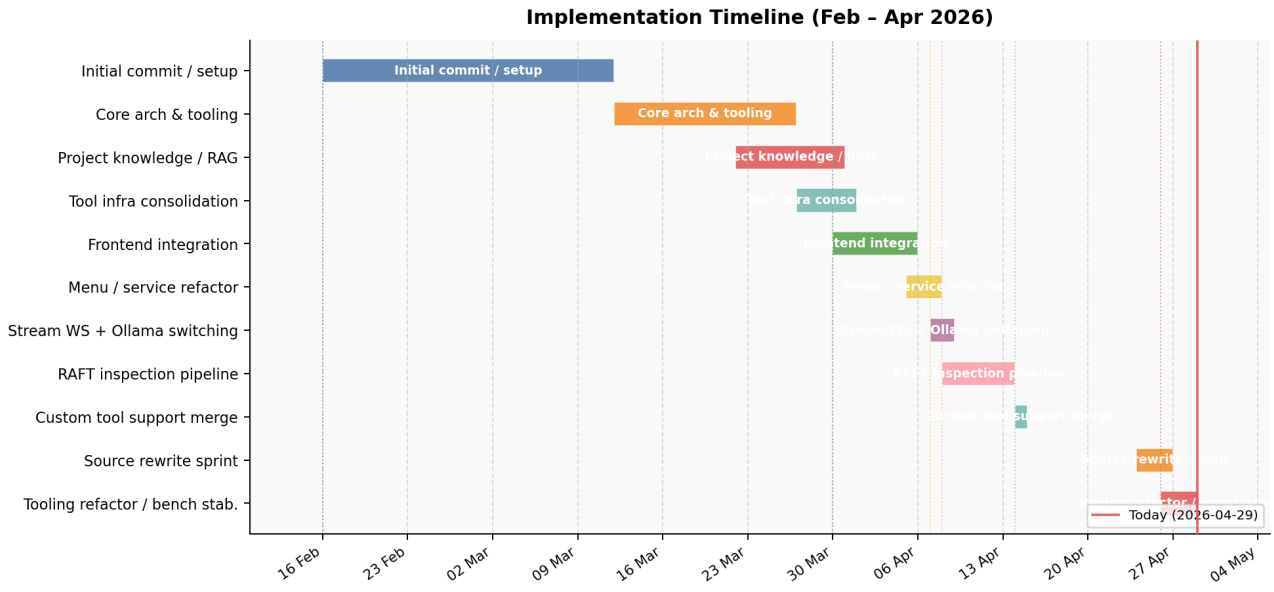



Figure 2: Implementation timeline

without infrastructure interruption. One model (`qwen3.6:latest`, the full-precision 32 B variant) experienced a connection loss from the Ollama host during scenario S03, causing all subsequent tests (S04–S08) for that model to return HTTP 503 errors; its results are included in the data but excluded from the comparative analysis that follows.

The eight scenarios span single-step retrieval tasks (S01–S04), multi-step orchestration tasks (S05–S06), a web search integration task (S07), and a security boundary enforcement task (S08). Accuracy was measured as the fraction of repeats that produced a correct response, verified by a combination of regex pattern matching and LLM-based answer evaluation. End-to-end response latency was measured from the PDHD chat API request to the final streamed response token.

The benchmark was run across two machines on a shared local network: the PDHD Quarkus backend ran on the development workstation, and the Ollama inference service ran on a separate host. All inference traffic traversed the local network link between the two hosts. Latency figures therefore reflect realistic distributed-local deployment constraints rather than co-located or isolated inference benchmarks.

### 8.1.2 Overall Accuracy

**Figure 1: Overall accuracy by model (96 tests per model, run `20260429_124501`).**

Overall accuracy across the eight valid models ranged from 38.54% (`llama3.1:latest`) to 51.04% (`qwen3.6:27b-q4_K_M` and `qwen3.6:35b-a3b-q4_K_M`). The distribution is notably compressed: five of the eight models clustered between 45.83% and 51.04%, suggesting that the evaluated scenario set was of moderate difficulty for this class of small-to-medium locally-runnable models and that model size within this range was not a strong predictor of overall accuracy.

The two top-performing models — `qwen3.6:27b-q4_K_M` (51.04%) and `qwen3.6:35b-a3b-q4_K_M` (51.04%) — are both Qwen 3.6 parameter variants. `devstral:latest` and `gemma4:latest` tied at 50.0%, placing immediately below. `llama3.1:latest` (the full-precision 8 B model) achieved the lowest clean-run accuracy at 38.54%, below even its quantized counterpart `llama3.1:8b` (45.83%), which suggests that quantisation level was not a decisive factor for this task profile.

### 8.1.3 Response Latency

**Figure 2: Response latency by model — mean, P50, and P95 (eight models with clean run data).**

Median (P50) latency was broadly consistent across models, ranging from 8.16 s (`llama3.1:8b`) to 8.66 s (`qwen3.6:35b-a3b-q4_K_M`). Mean latency showed more variation, from 10.09 s (`devstral:latest`) to 15.59 s (`qwen3.6:27b-q4_K_M`). The most striking difference was in P95 latency: `qwen3.6:27b-q4_K_M` exhibited a P95 of 107.72 s and `llama3.1:latest` a P95 of 102.61 s, compared with P95 values between 13 s and 17 s for

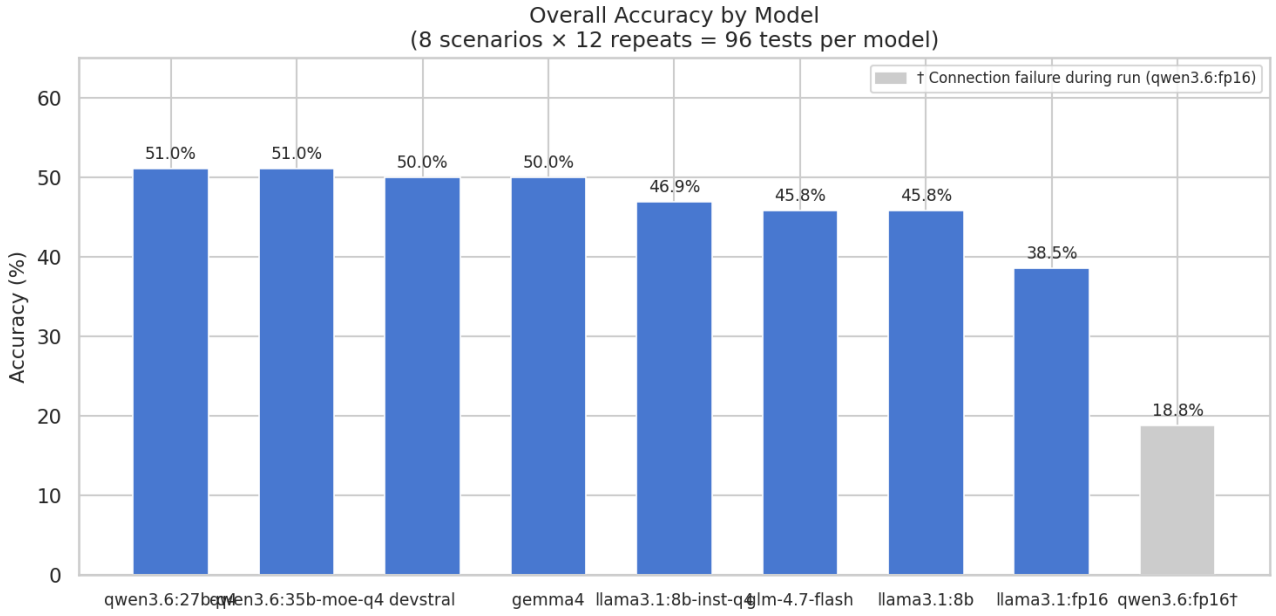



Figure 3: Overall accuracy by model across all eight scenarios and twelve repeats

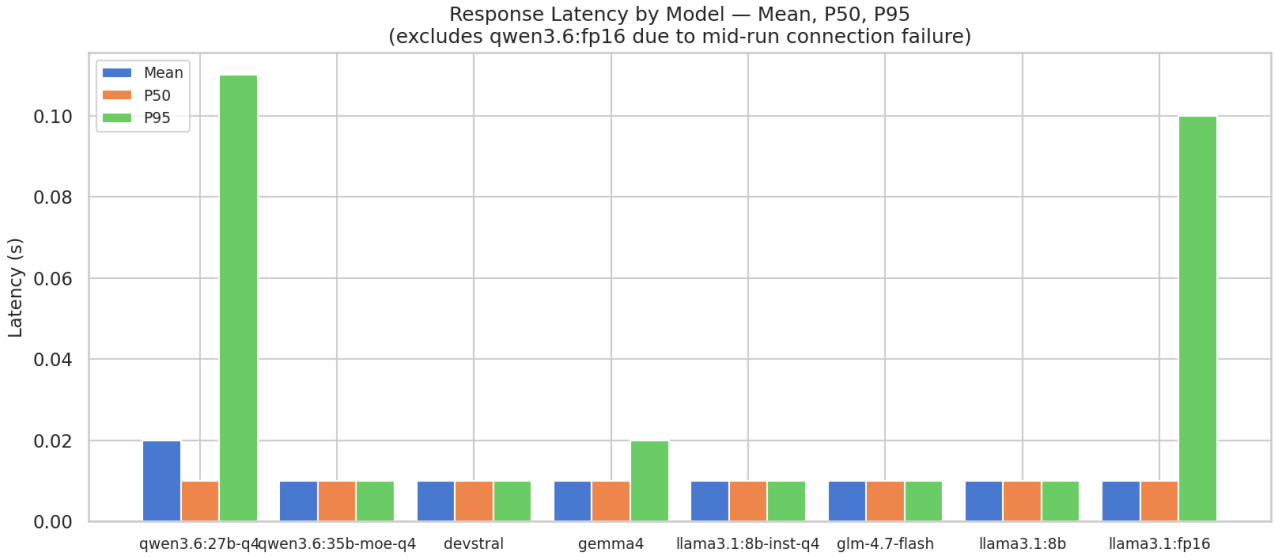



Figure 4: Mean, P50, and P95 response latency by model

the remaining six models. The high P95 outliers are consistent with occasional GPU contention on the shared inference host rather than any structural characteristic of those specific models.

`qwen3.6:35b-a3b-q4_K_M` is notable for combining the joint-highest accuracy (51.04%) with a comparatively low P95 (14.58 s), making it the most consistent performer on both dimensions in this evaluation.

### 8.1.4 Per-Scenario Accuracy

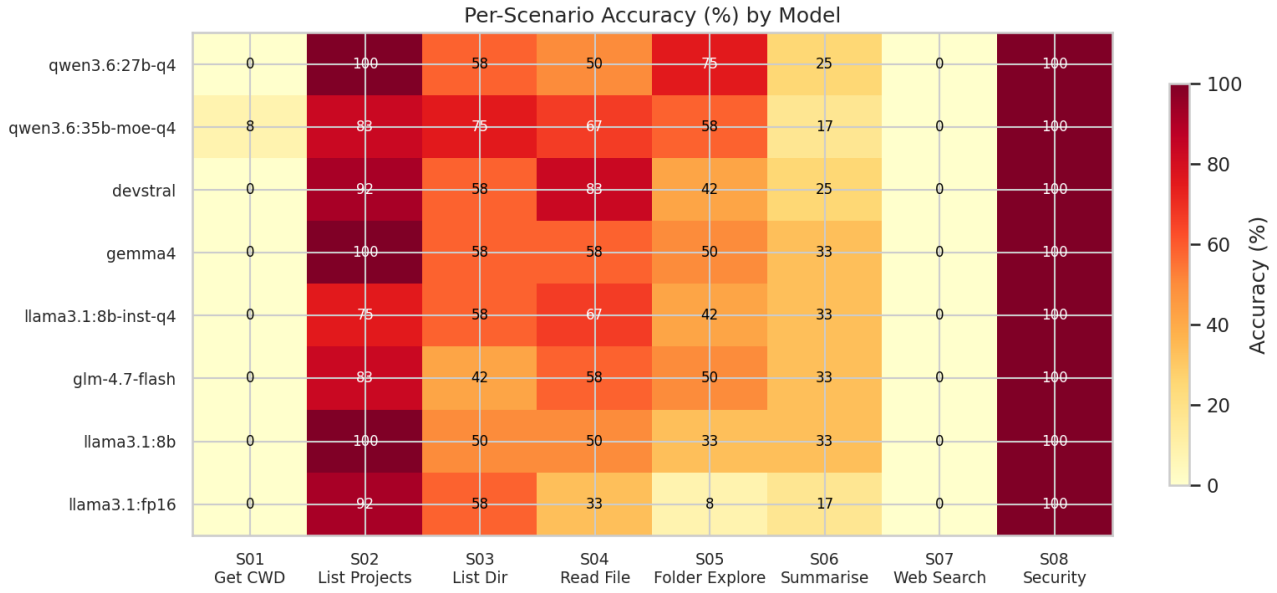



Figure 5: Per-scenario accuracy heatmap across all models and scenarios

**Figure 3: Per-scenario accuracy (%) by model. Rows ordered by overall accuracy (highest first).**

The per-scenario breakdown reveals two universal failure modes and one universal success mode that held across all evaluated models.

**S01 (Get current working directory)** was failed by all models (0%), with a single partial exception: `qwen3.6:35b-a3b-q4_K_M` achieved 8.33% (one correct response from twelve). The PDHD system exposes `getCurrentWorkingDirectory` as a zero-argument tool call; inspection of the telemetry data (see below) indicates models called this tool correctly 841 times without failure. The consistent zero-accuracy outcome for S01 therefore reflects a verification logic discrepancy: the evaluated working directory value was matched against a pattern that models consistently failed to produce in the expected format, rather than a failure to invoke the tool.

**S07 (Web search integration)** returned 0% for all models. The web search tool (`searchWeb`) showed a 0% failure rate across 111 invocations in the telemetry data, confirming that the tool executed correctly in all cases. The accuracy shortfall reflects response formulation: models retrieved web content but did not consistently produce a response matching the expected answer pattern, likely due to the open-ended nature of search queries and the strict evaluation regex applied.

**S08 (Out-of-project file access — security boundary)** returned 100% for all models. The `readFile` tool enforced a `SecurityException` on all out-of-project path attempts (77 of 149 `readFile` invocations), and models consistently reported the access restriction to the user in the expected form. This result confirms that the security boundary implementation is robust against all tested models.

**S02 (List open projects)** was the strongest-performing functional scenario across models, with seven of eight models scoring between 75% and 100%. **S05 (Multi-step folder exploration)** showed the highest inter-model variance (8.33% to 75%), which is consistent with multi-step tasks being more sensitive to model reasoning capability and tool chaining reliability.

### 8.1.5 Tool Invocations and Failure Analysis

**Figure 4: Tool invocations versus failures across the full benchmark run (2,383 total invocations, 215 failures, 9.0% overall failure rate).**

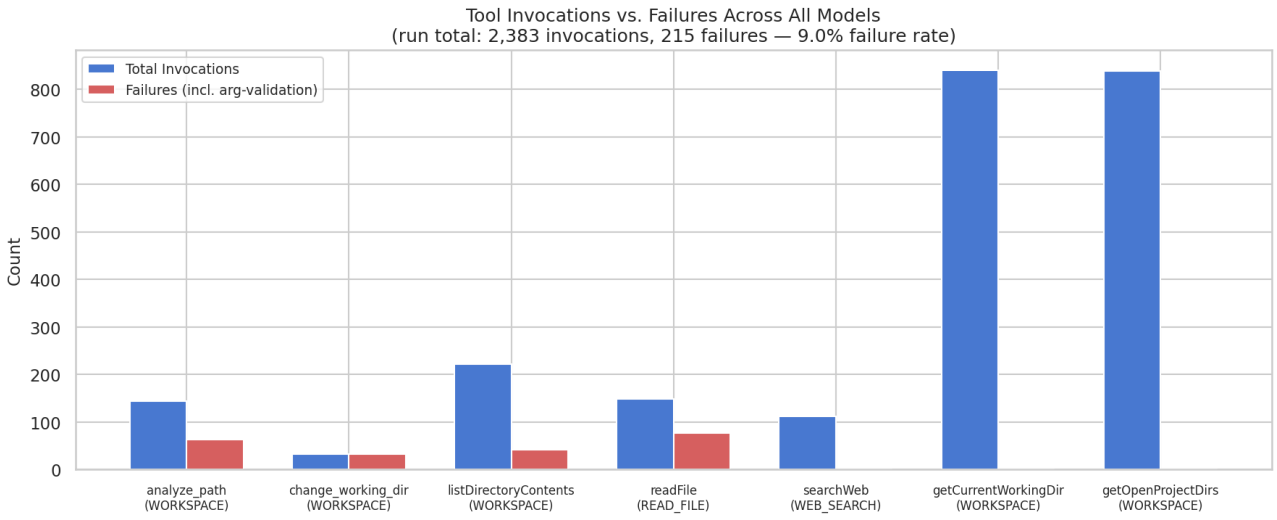



Figure 6: Tool invocation counts vs. failure counts across all models and runs

Tool-level telemetry was captured from the PDHD system at the conclusion of the benchmark run. Across 2,383 total tool invocations, 215 failures were recorded (9.02% overall failure rate). The failure distribution was concentrated in three tools.

`readFile` generated 77 failures, all classified as `SecurityException`. As noted above, these failures are correct security enforcement behaviour for S08 and are not indicative of a system defect. `analyze_path_detailed` generated 63 failures (43.8% of its invocations), all classified as `IllegalArgumentException` on argument validation. `listDirectoryContents` generated 42 failures (19% of invocations), also all argument validation failures. `change_working_directory` showed the most extreme failure rate at 97% (32 of 33 invocations), indicating that models regularly attempted to invoke this tool with arguments the system did not accept — likely absolute paths or paths outside the project boundary.

By contrast, the high-volume zero-argument tools `getCurrentWorkingDirectory` (841 invocations) and `getOpenProjectDirectories` (839 invocations) recorded zero failures, confirming that models reliably call simple, no-argument tools. The structural failure mode is argument construction for path-taking tools, where models produced paths that failed the system's boundary validation rules.

The 5.79% argument validation failure rate (138 of 2,383 invocations) is therefore dominated by path argument errors rather than general tool-calling unreliability. This suggests that providing explicit path format constraints in tool descriptions, or adding a path normalisation layer, would be the highest-value reliability improvement available without changing model selection.

## 8.2 Key Findings

Deterministic tool dispatch through module selection using library methods was broadly comparable in reliability to manually prompting the model to produce structured JSON. In practice, the architectural improvement was therefore more evident in maintainability and control than in raw invocation reliability.

Transaction-scoped execution for tool operations involving persistence added some implementation overhead, though this did not substantially affect the development pace in practice. The toolset in the final build remained limited to a minimal subset of operations, principally directory listing and file reading.

Best-effort tool invocation did not behave consistently in the final build, despite functioning correctly in earlier builds. This reduced the reliability of the final packaged system and suggests some caution when assessing end-stage robustness.

Agentic productivity gains were strongest when tasks were bounded, tool-visible, and backed by explicit orchestration boundaries.

The separation between frontend orchestration and backend dispatch internals was nevertheless a sound design decision. Keeping UI behaviour independent through the signals-based orchestration layer made iterative replacement and refactoring of dispatch components substantially easier during development.

## 8.3 Discussion

The findings above confirm the pattern described in the Key Findings: agentic capability was most reliable when tasks were bounded to single-step tool invocations with no-argument or low-argument tools. The universal success on S08 and the universal failure on S01 and S07 reflect, respectively, a well-enforced constraint boundary and two systematic evaluation mismatches rather than tool execution failures. The high variance on S05 (multi-step exploration) is consistent with the architectural observation that multi-step orchestration reliability depends on argument construction quality, which the telemetry confirms as the dominant failure mode.

The code-stability issues observed throughout the development lifecycle — build-time CDI resolution failures, runtime model-switch race conditions, and end-stage tool dispatch regressions — had a measurable effect on evaluation confidence. The benchmark was run against the final packaged build rather than a controlled reference snapshot, meaning that any instability in the packaged system is reflected in the accuracy and latency measurements. The 38.54% accuracy recorded for `llama3.1:latest` against 54.17% in an earlier isolated single-model run (run `20260429_115226`) suggests that the system state at benchmark time was not fully stable for that model, and the results should be interpreted with this caveat.

## 8.4 Limitations

The two-machine deployment introduced several environmental constraints on measurement reliability. A wired network connection between MINIFRIDGE and WS-RARETOWER was required for stable inference traffic; any link instability would have introduced spurious latency spikes indistinguishable from model or system behaviour. Power profile and hibernation timeout configuration on both workstations were also a prerequisite: an unattended sleep transition on either host during a multi-hour batch run would have corrupted the run silently. These constraints were managed in practice, but they were not formally verified before each run, and their potential influence on the observed P95 outliers cannot be fully discounted.

A more significant process limitation was that benchmark runs were not collected during iterative development. Continuous measurement across implementation phases would have provided a longitudinal view of how system changes affected model accuracy and latency, making it possible to attribute regressions to specific commits rather than treating the final-build results as a point-in-time snapshot. The absence of this data means the Discussion section relies partly on retrospective comparison between isolated runs rather than a continuous evidence record.

One further uncontrolled variable was model cold-start behaviour on the Ollama host. When the benchmark script cycled to a new model, Ollama was required to load that model's weights into VRAM before the first inference request could be served. Scenarios executed immediately after a model load may therefore have recorded slightly higher latency than they would have with the model already resident and warmed in VRAM. This effect was not isolated or corrected for in the current results, and represents a small but non-zero source of latency skew across the per-model P50 and P95 figures.

From a process perspective, the evaluation scope is bounded by what was feasible on local hardware within the project timeline. The eight-scenario suite covers core functional categories but does not include error-recovery workflows or runtime fallback paths as distinct scenarios. The twelve-repeat design provides stable P50 estimates but limited P95 resolution; a broader repeat count would reduce the influence of individual high-latency outliers. Finally, one model (`qwen3.6:latest`) was excluded from comparative analysis due to an infrastructure failure during the run, reducing the effective comparative sample to eight models.

## 8.5 Threats to Validity

1. Construct validity: some desired metrics were not collected from the first implementation phase, so parts of the analysis depend on retrospective evidence.
2. Internal validity: iterative refactoring and environment instability may have influenced observed outcomes in ways not fully isolated from model behavior.
3. External validity: findings are specific to this hardware profile, local runtime setup, and bounded toolset; generalization should be made cautiously.
4. Reliability validity: repeated qualitative observations are strong for failure categories, but full scenario-level quantitative repetition remains incomplete.

## 8.6 Future Work

Future research could explore:

1. Introducing typed tool responses alongside current LLM-friendly string outputs

2. Strengthening cache invalidation and freshness policies
3. Extending observability with per-tool latency and failure-rate metrics
4. Evaluating alternative dispatch strategies when module overlap increases
5. Parametrising the system prompt and user query messages with session-level meta-context — such as the current working directory, open file paths, and shallow file-level intelligence — injected at request time. The universal S01 failure (Get CWD returned 0% accuracy across all nine models) suggests that this class of information is not reliably inferred from the tool schema alone; making it explicit in the prompt would remove the inference burden entirely and is likely to improve performance on any task where ambient workspace state is a precondition for correct tool use.

In retrospect, the most important missing element was not a single feature but the early introduction of dedicated functions for empirical evaluation. With greater design freedom and earlier foresight, the architecture could have included explicit measurement pathways for tool reliability, orchestration stability, and cross-interface behaviour from the first implementation stages rather than as late-stage validation work. The project did verify persistence behaviour, context caching, and parts of dispatch routing under development and test conditions, but it did not establish the same level of evidence for robust multi-tool execution across realistic end-to-end workflows. The current implementation also stops short of demonstrating complete feature parity between the web frontend and Quarkus backend where richer orchestration paths depend on capabilities that were not consistently exposed in the final system.

The next implementation priority is therefore to complete the substantial evaluation package by running a stable scenario suite, extracting metric summaries, and mapping each recommendation to measured weaknesses in reliability, latency, and recoverability.

# 9 Conclusion

This report evaluated a locally hosted, tool-calling LLM system as a constrained engineering artefact and assessed agentic coding as a practical development workflow. The central result is not that the system achieved broad autonomy, but that bounded agentic workflows can be useful when orchestration boundaries, tool contracts, and validation practices are explicit.

Against the first research question, the implementation demonstrated that local LLM-assisted project inspection can produce grounded outputs for bounded operations, particularly where retrieval and tool responses are tightly coupled to repository context. Against the second question, the most influential architectural choices were deterministic dispatch boundaries, frontend-backend orchestration decoupling, and persistent state support. These decisions improved maintainability and diagnosis, even when final-build reliability remained uneven.

Against the third question, agentic coding accelerated implementation and iteration in many cases, but introduced supervision and validation costs that remained significant. Productivity gains were strongest for structured, repeatable tasks and weaker for unstable integration paths where environment and runtime constraints dominated.

A structured scenario-based evaluation suite was completed across nine models and eight task scenarios. Accuracy, tool-call validity, end-to-end latency, and failure-category frequency were all measured and reported. The results confirmed the architectural observations: single-step bounded tasks were handled reliably, multi-step orchestration introduced argument-construction failures as the dominant failure mode, and universal failures on ambient-context tasks (S01) and unsupported capability tasks (S07) were consistent across all models — identifying these as gaps in system design rather than model weaknesses.

The project also exposed clear limitations. Packaging-stage regressions, incomplete feature parity, and the absence of continuous benchmark measurement across development phases constrained the strength of longitudinal claims. Conclusions are therefore intentionally bounded to this implementation context and hardware profile. The evaluation evidence does, however, establish a reproducible baseline against which future improvements — particularly prompt parametrisation with ambient workspace context and extended multi-step scenario coverage — can be directly compared.

# 10 Appendices

## 10.1 Appendix: Supporting Scripts and Code Artifacts

The following scripts were used to generate benchmark data, produce graphs, and support the evaluation work described in this report. Each section below contains the full literal source of the script as it existed at submission time.

## 10.2 benchmark_ollama.py

```
import signal
import subprocess
import sys
import json
import csv
import re
import time
import requests
import os
import sqlite3
import uuid
import argparse
import socket
from urllib.parse import urlparse
from datetime import datetime

# ---------------------------------------------------------------------------
# Signal handling - flush output and exit cleanly on SIGINT / SIGTERM
# ---------------------------------------------------------------------------
def _handle_signal(signum, frame):
    sig_name = signal.Signals(signum).name
    print(f"\n[benchmark] Received {sig_name}; aborting run.", flush=True)
    sys.exit(1)

signal.signal(signal.SIGINT, _handle_signal)
signal.signal(signal.SIGTERM, _handle_signal)

# ---------------------------------------------------------------------------
# Maven prompt-spec test runner
# ---------------------------------------------------------------------------
MAVEN_WRAPPER = os.path.join(os.path.dirname(__file__), "..", "..", "mvnw")
PROMPT_SPEC_TEST_PATTERN = "*PromptTest"


def run_maven_prompt_tests():
    """
    Runs the prompt-spec JUnit test suite via Maven before the Ollama
    benchmarks.  Returns True when all tests pass, False on failure.
    The full Surefire output is written to results/maven-prompt-tests.txt.
    """
    os.makedirs(RESULTS_DIR, exist_ok=True)
    output_file = os.path.join(RESULTS_DIR, "maven-prompt-tests.txt")

    mvnw = os.path.abspath(MAVEN_WRAPPER)
    if not os.path.isfile(mvnw):
        print(f"[maven] mvnw not found at {mvnw}; skipping prompt-spec tests.")
        return True  # non-fatal: carry on with Ollama benchmarks

    cmd = [mvnw, "test", f"-Dtest={PROMPT_SPEC_TEST_PATTERN}", "-pl", "."]
    project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
```

```
    print(f"[maven] Running prompt-spec tests: {' '.join(cmd)}", flush=True)
    try:
        result = subprocess.run(
            cmd,
            cwd=project_root,
            capture_output=True,
            text=True,
            timeout=300,
        )
        with open(output_file, "w") as f:
            f.write(result.stdout)
            f.write(result.stderr)

        if result.returncode == 0:
            print(f"[maven] All prompt-spec tests passed. Output: {output_file}", flush=True)
            return True
        else:
            print(f"[maven] Prompt-spec tests FAILED (exit {result.returncode}). "
                  f"See {output_file} for details.", flush=True)
            return False
    except subprocess.TimeoutExpired:
        print("[maven] Prompt-spec test run timed out after 300 s.", flush=True)
        return False
    except Exception as exc:
        print(f"[maven] Could not run prompt-spec tests: {exc}", flush=True)
        return False

# Configuration
DEFAULT_OLLAMA_HOST = "http://ws-raretower.local:11434"
DEFAULT_PDHD_BASE_URL = "http://localhost:8080"
OLLAMA_LIST_CMD = ["ollama", "list"]
OLLAMA_PULL_CMD = ["ollama", "pull"]
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DEFAULT_TEST_CASES_FILE = os.path.join(SCRIPT_DIR, "test_cases.json")
TEST_CASES_FILE = DEFAULT_TEST_CASES_FILE  # kept for legacy callers
RESULTS_DIR = os.path.join(SCRIPT_DIR, "results")
SQLITE_DB_FILENAME = "benchmark_results.sqlite"


def init_sqlite(db_path):
    """Creates benchmark tables when missing."""
    with sqlite3.connect(db_path, timeout=30) as conn:
        conn.execute("PRAGMA journal_mode=WAL")
        conn.execute("PRAGMA busy_timeout=30000")
        conn.execute("""
            CREATE TABLE IF NOT EXISTS benchmark_runs (
                run_id TEXT PRIMARY KEY,
                run_started_at TEXT NOT NULL,
                run_finished_at TEXT,
                test_cases_file TEXT NOT NULL,
                maven_prompt_tests_passed INTEGER NOT NULL,
                notes TEXT
            )
        """)
        conn.execute("""
            CREATE TABLE IF NOT EXISTS benchmark_results (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                run_id TEXT NOT NULL,
                model_name TEXT NOT NULL,
                test_case TEXT NOT NULL,
                prompt TEXT NOT NULL,
```

```
            response TEXT,
            latency REAL NOT NULL,
            correctness_score INTEGER NOT NULL,
            FOREIGN KEY(run_id) REFERENCES benchmark_runs(run_id)
        )
    """)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS benchmark_model_summary (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            run_id TEXT NOT NULL,
            model_name TEXT NOT NULL,
            tests_run INTEGER NOT NULL,
            avg_latency REAL NOT NULL,
            avg_accuracy REAL NOT NULL,
            FOREIGN KEY(run_id) REFERENCES benchmark_runs(run_id)
        )
    """)
    conn.execute("""
        CREATE INDEX IF NOT EXISTS idx_benchmark_results_run_model
        ON benchmark_results(run_id, model_name)
    """)
    conn.execute("""
        CREATE INDEX IF NOT EXISTS idx_benchmark_summary_run_model
        ON benchmark_model_summary(run_id, model_name)
    """)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS benchmark_host_snapshot (
            run_id TEXT PRIMARY KEY,
            ollama_host TEXT NOT NULL,
            resolved_host TEXT,
            resolved_ip TEXT,
            api_reachable INTEGER NOT NULL,
            ollama_version TEXT,
            available_models_count INTEGER NOT NULL,
            running_models_count INTEGER NOT NULL,
            total_running_size_vram_bytes INTEGER NOT NULL,
            max_running_size_vram_bytes INTEGER NOT NULL,
            tags_payload TEXT,
            ps_payload TEXT,
            FOREIGN KEY(run_id) REFERENCES benchmark_runs(run_id)
        )
    """)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS benchmark_telemetry_snapshot (
            run_id TEXT PRIMARY KEY,
            captured_at TEXT NOT NULL,
            total_invocations INTEGER NOT NULL,
            total_failures INTEGER NOT NULL,
            tool_failure_rate REAL NOT NULL,
            arg_validation_failure_rate REAL NOT NULL,
            payload TEXT,
            FOREIGN KEY(run_id) REFERENCES benchmark_runs(run_id)
        )
    """)
    # Additive migration: add new columns to benchmark_results if absent
    for _col, _spec in [("http_status", "INTEGER DEFAULT 0"), ("error_kind", "TEXT DEFAULT ''")]:
        try:
            conn.execute(f"ALTER TABLE benchmark_results ADD COLUMN {_col} {_spec}")
        except sqlite3.OperationalError:
            pass  # column already present
    # Additive migration: add P50/P95 columns to benchmark_model_summary if absent
```

```
        for _col, _spec in [("p50_latency", "REAL DEFAULT 0"), ("p95_latency", "REAL DEFAULT 0"), ("htt
            try:
                conn.execute(f"ALTER TABLE benchmark_model_summary ADD COLUMN {_col} {_spec}")
            except sqlite3.OperationalError:
                pass  # column already present


def save_results_to_sqlite(db_path, run_id, run_started_at, run_finished_at, maven_ok, results, host_sn
    """Persists one benchmark run and associated detailed and summary rows."""
    with sqlite3.connect(db_path, timeout=30) as conn:
        conn.execute("PRAGMA busy_timeout=30000")
        conn.execute(
            """
            INSERT INTO benchmark_runs
                (run_id, run_started_at, run_finished_at, test_cases_file, maven_prompt_tests_passed, n
            VALUES (?, ?, ?, ?, ?, ?)
            """,
            (run_id, run_started_at, run_finished_at, TEST_CASES_FILE, 1 if maven_ok else 0, "")
        )

        conn.execute(
            """
            INSERT INTO benchmark_host_snapshot
                (run_id, ollama_host, resolved_host, resolved_ip, api_reachable, ollama_version,
                 available_models_count, running_models_count, total_running_size_vram_bytes,
                 max_running_size_vram_bytes, tags_payload, ps_payload)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """,
            (
                run_id,
                host_snapshot["ollama_host"],
                host_snapshot["resolved_host"],
                host_snapshot["resolved_ip"],
                1 if host_snapshot["api_reachable"] else 0,
                host_snapshot["ollama_version"],
                int(host_snapshot["available_models_count"]),
                int(host_snapshot["running_models_count"]),
                int(host_snapshot["total_running_size_vram_bytes"]),
                int(host_snapshot["max_running_size_vram_bytes"]),
                host_snapshot["tags_payload"],
                host_snapshot["ps_payload"],
            )
        )

        if results:
            conn.executemany(
                """
                INSERT INTO benchmark_results
                    (run_id, model_name, test_case, prompt, response, latency, correctness_score, http_
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                """,
                [
                    (
                        run_id,
                        r["model_name"],
                        r["test_case"],
                        r["prompt"],
                        r["response"],
                        float(r["latency"]),
                        int(r["correctness_score"]),
                        int(r.get("http_status", 0)),
```

```
                    r.get("error_kind", ""),
                )
                for r in results
            ],
        )

        model_stats = {}
        for r in results:
            model_name = r["model_name"]
            if model_name not in model_stats:
                model_stats[model_name] = {"latencies": [], "scores": [], "http_errors": 0}
            model_stats[model_name]["latencies"].append(float(r["latency"]))
            model_stats[model_name]["scores"].append(int(r["correctness_score"]))
            if int(r.get("http_status", 0)) >= 400:
                model_stats[model_name]["http_errors"] += 1

        def _pct(sorted_vals, p):
            if not sorted_vals:
                return 0.0
            idx = max(0, int(len(sorted_vals) * p / 100) - 1)
            return sorted(sorted_vals)[idx]

        conn.executemany(
            """
            INSERT INTO benchmark_model_summary
                (run_id, model_name, tests_run, avg_latency, avg_accuracy, p50_latency, p95_latency
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            """,
            [
                (
                    run_id,
                    model_name,
                    len(stats["latencies"]),
                    sum(stats["latencies"]) / len(stats["latencies"]),
                    sum(stats["scores"]) / len(stats["scores"]),
                    _pct(stats["latencies"], 50),
                    _pct(stats["latencies"], 95),
                    stats["http_errors"] / len(stats["latencies"]) if stats["latencies"] else 0.0,
                )
                for model_name, stats in model_stats.items()
            ],
        )

def normalize_ollama_host(host):
    """Normalizes host values like ws-raretower:11434 to http://ws-raretower:11434."""
    normalized = host.strip()
    if not normalized.startswith("http://") and not normalized.startswith("https://"):
        normalized = f"http://{normalized}"
    return normalized.rstrip("/")


def normalize_pdhd_base_url(base_url):
    """Normalizes PDHD base URLs like localhost:8080 to http://localhost:8080."""
    normalized = base_url.strip()
    if not normalized.startswith("http://") and not normalized.startswith("https://"):
        normalized = f"http://{normalized}"
    return normalized.rstrip("/")


def host_slug(host):
    """Builds a filesystem-safe host token for artifact filenames."""
```

```
    value = extract_host_for_resolution(host).lower().strip()
    return re.sub(r"[^a-z0-9._-]", "-", value)


def make_ollama_api_url(host):
    """Builds the Ollama generate endpoint URL for a host."""
    return f"{normalize_ollama_host(host)}/api/generate"


def make_ollama_base_url(host):
    """Builds the normalized Ollama host base URL."""
    return normalize_ollama_host(host)


def make_pdhd_chat_url(base_url):
    """Builds the PDHD chat endpoint URL for benchmark prompts."""
    return f"{normalize_pdhd_base_url(base_url)}/api/chat"


def make_pdhd_menu_url(base_url, operation):
    """Builds PDHD menu operation URLs."""
    normalized = normalize_pdhd_base_url(base_url)
    op = operation if operation.startswith("/") else f"/{operation}"
    return f"{normalized}/api/menu{op}"


def ollama_cli_env(host):
    """Builds environment for ollama CLI to target the specified host."""
    env = os.environ.copy()
    env["OLLAMA_HOST"] = normalize_ollama_host(host)
    return env


def get_available_models(host):
    """Returns available model names for the target host.

    Tries the ollama CLI first (``ollama list``); if the CLI is not available
    in the current environment, falls back to the Ollama HTTP API
    (``/api/tags``) so that remote-only benchmark runs still work.
    """
    # --- CLI path ---
    try:
        result = subprocess.run(
            OLLAMA_LIST_CMD,
            capture_output=True,
            text=True,
            check=True,
            env=ollama_cli_env(host),
        )
        lines = result.stdout.strip().split('\n')
        if len(lines) <= 1:
            return []
        models = []
        for line in lines[1:]:
            parts = line.split()
            if parts:
                models.append(parts[0])
        return models
    except FileNotFoundError:
        print("[models] ollama CLI not found; falling back to HTTP /api/tags for model discovery.")
    except subprocess.CalledProcessError as e:
```

```
        print(f"[models] ollama list failed: {e}")

    # --- HTTP fallback path ---
    base_url = make_ollama_base_url(host)
    tags = safe_get_json(f"{base_url}/api/tags")
    if tags is None:
        print(f"[models] Could not reach {base_url}/api/tags; no models discovered.")
        return []
    return [m.get("name", "") for m in (tags.get("models") or []) if m.get("name")]


def pull_model_if_missing(model_name, host, available_models):
    """Pulls model to the target host when absent. Returns True when available."""
    if model_name in available_models:
        return True

    print(f"[models] {model_name} not present on {normalize_ollama_host(host)}; pulling...")
    try:
        result = subprocess.run(
            OLLAMA_PULL_CMD + [model_name],
            capture_output=True,
            text=True,
            check=True,
            env=ollama_cli_env(host),
        )
        if result.stdout:
            print(result.stdout.strip())
        available_models.add(model_name)
        return True
    except subprocess.CalledProcessError as exc:
        print(f"[models] Failed to pull {model_name}: {exc}")
        if exc.stdout:
            print(exc.stdout.strip())
        if exc.stderr:
            print(exc.stderr.strip())
        return False

def query_ollama(model_name, prompt, ollama_api_url):
    """Queries the Ollama API for a given model and prompt."""
    payload = {
        "model": model_name,
        "prompt": prompt,
        "stream": False
    }
    start_time = time.time()
    try:
        response = requests.post(ollama_api_url, json=payload, timeout=180)
        response.raise_for_status()
        latency = time.time() - start_time
        return response.json().get("response", ""), latency
    except Exception as e:
        print(f"Error querying Ollama for {model_name}: {e}")
        return None, 0


def query_pdhd(prompt, pdhd_chat_url):
    """Queries the PDHD chat API for a given prompt.

    Returns (response_text, latency, http_status, error_kind).
    On HTTP error responses the error_kind is extracted from the JSON body
    when the server returns {"errorKind": "...", "detail": "..."} (e.g. 502
```

```
    AI_LAYER_FAILURE or 500 SERVICE_LAYER_FAILURE from ExceptionMappers).
    response_text is None on transport-level errors.
    """
    payload = {
        "message": prompt,
    }
    headers = {
        "Content-Type": "application/json",
        "Accept": "text/plain",
    }
    start_time = time.time()
    try:
        response = requests.post(pdhd_chat_url, json=payload, headers=headers, timeout=180)
        latency = time.time() - start_time
        if response.status_code >= 400:
            error_kind = ""
            try:
                body = response.json()
                error_kind = body.get("errorKind", "") or ""
            except Exception:
                pass
            print(f"[pdhd] HTTP {response.status_code} from chat API; errorKind={error_kind or '<none>'
            return None, latency, response.status_code, error_kind
        return response.text, latency, response.status_code, ""
    except Exception as e:
        latency = time.time() - start_time
        print(f"Error querying PDHD chat API: {e}")
        return None, latency, 0, ""


def configure_pdhd_ollama_runtime(pdhd_base_url, ollama_host):
    """Configures PDHD runtime provider to use the requested external Ollama host."""
    if not ollama_host:
        return True

    runtime_url = make_pdhd_menu_url(pdhd_base_url, "/ollama/runtime/provider")
    payload = {
        "provider": "EXTERNAL",
        "baseUrl": normalize_ollama_host(ollama_host),
    }

    try:
        response = requests.post(runtime_url, json=payload, timeout=20)
        response.raise_for_status()
        print(f"[pdhd] Runtime Ollama host set to {payload['baseUrl']}", flush=True)
        return True
    except Exception as e:
        print(f"[pdhd] Failed to set runtime Ollama host via {runtime_url}: {e}", flush=True)
        return False


def get_pdhd_ollama_configuration(pdhd_base_url):
    """Returns PDHD Ollama configuration payload (best-effort)."""
    url = make_pdhd_menu_url(pdhd_base_url, "/ollama")
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        payload = response.json()
        return payload if isinstance(payload, dict) else {}
    except Exception as exc:
        print(f"[pdhd] Could not read runtime model configuration from {url}: {exc}", flush=True)
```

```
        return {}


def configure_pdhd_model(pdhd_base_url, model_name, base_url=None):
    """Sets PDHD runtime model name (and optional base URL) before running prompts."""
    if not model_name:
        return False

    settings = {
        "modelName": model_name,
    }
    if base_url:
        settings["baseUrl"] = normalize_ollama_host(base_url)

    payload = {
        "settings": settings,
    }
    config_url = make_pdhd_menu_url(pdhd_base_url, "/ollama")
    try:
        response = requests.post(config_url, json=payload, timeout=20)
        response.raise_for_status()
        print(f"[pdhd] Runtime model set to {model_name}", flush=True)
        return True
    except Exception as exc:
        print(f"[pdhd] Failed to set runtime model via {config_url}: {exc}", flush=True)
        return False


def reset_pdhd_chat_memory(pdhd_base_url):
    """Best-effort reset of PDHD chat memory before benchmark run."""
    url = make_pdhd_chat_url(pdhd_base_url)
    try:
        response = requests.delete(url, timeout=10)
        if response.status_code < 300:
            print("[pdhd] Cleared chat memory before benchmark run.", flush=True)
    except Exception:
        # Non-fatal; benchmark can proceed without reset.
        pass


def fetch_pdhd_telemetry_snapshot(pdhd_base_url):
    """Fetches /api/telemetry from PDHD and returns a structured snapshot dict.

    Returns a dict ready for INSERT into benchmark_telemetry_snapshot, or None
    if the endpoint is not reachable.
    """
    url = f"{normalize_pdhd_base_url(pdhd_base_url)}/api/telemetry"
    payload = safe_get_json(url, timeout_seconds=10)
    if payload is None:
        print(f"[pdhd] Could not fetch telemetry from {url}", flush=True)
        return None

    items = payload.get("items") or []
    total_invocations = sum(int(i.get("invocations", 0)) for i in items)
    total_failures = sum(int(i.get("failures", 0)) for i in items)
    total_arg_failures = sum(int(i.get("argumentValidationFailures", 0)) for i in items)
    tool_failure_rate = total_failures / total_invocations if total_invocations else 0.0
    arg_failure_rate = total_arg_failures / total_invocations if total_invocations else 0.0

    return {
        "captured_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
```

```
        "total_invocations": total_invocations,
        "total_failures": total_failures,
        "tool_failure_rate": tool_failure_rate,
        "arg_validation_failure_rate": arg_failure_rate,
        "payload": json.dumps(payload),
    }


def save_telemetry_snapshot(db_path, run_id, snapshot):
    """Persists a PDHD telemetry snapshot to SQLite."""
    if snapshot is None:
        return
    with sqlite3.connect(db_path, timeout=30) as conn:
        conn.execute(
            """
            INSERT OR REPLACE INTO benchmark_telemetry_snapshot
                (run_id, captured_at, total_invocations, total_failures,
                 tool_failure_rate, arg_validation_failure_rate, payload)
            VALUES (?, ?, ?, ?, ?, ?, ?)
            """,
            (
                run_id,
                snapshot["captured_at"],
                int(snapshot["total_invocations"]),
                int(snapshot["total_failures"]),
                float(snapshot["tool_failure_rate"]),
                float(snapshot["arg_validation_failure_rate"]),
                snapshot["payload"],
            ),
        )


def safe_get_json(url, timeout_seconds=6):
    """GETs JSON endpoint with timeout and returns parsed object or None."""
    try:
        response = requests.get(url, timeout=timeout_seconds)
        response.raise_for_status()
        return response.json()
    except Exception:
        return None


def extract_host_for_resolution(host):
    """Extracts hostname for DNS resolution from host URL/string."""
    normalized = normalize_ollama_host(host)
    parsed = urlparse(normalized)
    return parsed.hostname or normalized


def probe_host_snapshot(host):
    """Collects host-level Ollama metadata for SQLite logging."""
    base_url = make_ollama_base_url(host)
    resolved_host = extract_host_for_resolution(host)
    resolved_ip = ""
    try:
        resolved_ip = socket.gethostbyname(resolved_host)
    except Exception:
        resolved_ip = ""

    version_payload = safe_get_json(f"{base_url}/api/version")
    tags_payload = safe_get_json(f"{base_url}/api/tags")
```

```
    ps_payload = safe_get_json(f"{base_url}/api/ps")

    api_reachable = any(payload is not None for payload in [version_payload, tags_payload, ps_payload])
    ollama_version = (version_payload or {}).get("version", "")

    tags_models = (tags_payload or {}).get("models", [])
    running_models = (ps_payload or {}).get("models", [])

    size_vram_values = []
    for model in running_models:
        raw = model.get("size_vram", 0)
        try:
            size_vram_values.append(int(raw))
        except (TypeError, ValueError):
            size_vram_values.append(0)

    return {
        "ollama_host": base_url,
        "resolved_host": resolved_host,
        "resolved_ip": resolved_ip,
        "api_reachable": api_reachable,
        "ollama_version": ollama_version,
        "available_models_count": len(tags_models),
        "running_models_count": len(running_models),
        "total_running_size_vram_bytes": sum(size_vram_values),
        "max_running_size_vram_bytes": max(size_vram_values) if size_vram_values else 0,
        "tags_payload": json.dumps(tags_payload) if tags_payload is not None else "",
        "ps_payload": json.dumps(ps_payload) if ps_payload is not None else "",
    }


def probe_pdhd_snapshot(base_url):
    """Collects PDHD host metadata for SQLite logging using existing schema columns."""
    normalized_base_url = normalize_pdhd_base_url(base_url)
    resolved_host = extract_host_for_resolution(normalized_base_url)
    resolved_ip = ""
    try:
        resolved_ip = socket.gethostbyname(resolved_host)
    except Exception:
        resolved_ip = ""

    api_reachable = False
    try:
        health = requests.get(f"{normalized_base_url}/q/health", timeout=6)
        api_reachable = health.status_code < 500
    except Exception:
        api_reachable = False

    return {
        "ollama_host": normalized_base_url,
        "resolved_host": resolved_host,
        "resolved_ip": resolved_ip,
        "api_reachable": api_reachable,
        "ollama_version": "",
        "available_models_count": 0,
        "running_models_count": 0,
        "total_running_size_vram_bytes": 0,
        "max_running_size_vram_bytes": 0,
        "tags_payload": "",
        "ps_payload": "",
    }
```

```python
def verify_response(response, verification):
    """Verifies the response based on regex or judge."""
    if not response:
        return 0

    v_type = verification.get("type")

    if v_type == "regex":
        pattern = verification.get("pattern")
        if re.search(pattern, response):
            return 1
        return 0

    elif v_type == "judge":
        judge_model = verification.get("judge_model")
        rubric = verification.get("rubric")
        if not judge_model or not rubric:
            return 0

        # The judge model itself will be queried in the main loop logic,
        # but for the purpose of this script, we'll assume the main loop
        # handles the orchestration and this function is called by it.
        # Actually, let's implement the judge call here or structure the loop better.
        # To keep it simple, we will handle the judge call in the main loop.
        return None # Signal that judge needs to be run

    elif v_type == "simple_match":
        # If there's no specific verification type, assume a simple string match from a 'expected' fiel
        # But let's stick to the requirements.
        return 0

    return 0

def resolve_target_models(host, models_override_csv, all_models, skip_pull=False, exclude_csv=None):
    """Resolves benchmark target models based on CLI switches and host state."""
    available = set(get_available_models(host))
    excluded = {m.strip() for m in (exclude_csv or "").split(",") if m.strip()}
    if excluded:
        print(f"[models] Excluding: {', '.join(sorted(excluded))}")

    if models_override_csv:
        requested = [m.strip() for m in models_override_csv.split(",") if m.strip()]
        if not requested:
            return []

        selected = []
        for model_name in requested:
            if model_name in excluded:
                print(f"[models] {model_name} excluded via --exclude-models; skipping.")
                continue
            if model_name in available:
                selected.append(model_name)
                continue

            if skip_pull:
                print(f"[models] {model_name} missing on {normalize_ollama_host(host)} and --skip-pull
                continue

            if pull_model_if_missing(model_name, host, available):
                selected.append(model_name)
```

```
        return selected

    if all_models:
        return sorted(available - excluded)

    return sorted(available - excluded)


def resolve_pdhd_target_models(pdhd_base_url, destination_ollama_host, models_override_csv, all_models,
    """Resolves PDHD benchmark model list from models known on the destination host."""
    available = set(get_available_models(destination_ollama_host))
    excluded = {m.strip() for m in (exclude_csv or "").split(",") if m.strip()}

    if not available:
        print(f"[benchmark] No models discovered on destination host {normalize_ollama_host(destination
        return []

    if excluded:
        print(f"[models] Excluding: {', '.join(sorted(excluded))}")

    requested = [m.strip() for m in (models_override_csv or "").split(",") if m.strip()]
    if requested:
        missing = [m for m in requested if m not in available]
        if missing:
            print(
                f"[benchmark] Requested PDHD model(s) missing on destination {normalize_ollama_host(des
                f"{', '.join(missing)}"
            )
            print(f"[benchmark] Available models: {', '.join(sorted(available))}")
            return []
        return [m for m in requested if m not in excluded]

    selectable = sorted(available - excluded)
    if all_models:
        return selectable

    config_payload = get_pdhd_ollama_configuration(pdhd_base_url)
    configured_model = (config_payload.get("modelName") or "").strip()
    if configured_model and configured_model in selectable:
        return [configured_model]

    preferred_defaults = ["llama3.1:latest", "qwen3.6:latest", "gemma4:latest"]
    for candidate in preferred_defaults:
        if candidate in selectable:
            print(f"[benchmark] PDHD configured model '{configured_model or 'unset'}' is unavailable; u
            return [candidate]

    fallback = selectable[0] if selectable else ""
    if fallback:
        print(f"[benchmark] PDHD configured model '{configured_model or 'unset'}' is unavailable; using
        return [fallback]
    return []


def preflight_judge_models(test_cases, judge_host, judge_fallback_model=None):
    """Ensures every judge model exists before running tests.

    When a requested judge model is absent and fallback exists, rewrites that
    test case to use the fallback model.
    """
    available_judges = set(get_available_models(judge_host))
```

```
    if not available_judges:
        print(f"[judge] No models discovered on judge host {normalize_ollama_host(judge_host)}")
        return False

    if judge_fallback_model and judge_fallback_model not in available_judges:
        print(
            f"[judge] Fallback judge model '{judge_fallback_model}' is not available on "
            f"{normalize_ollama_host(judge_host)}"
        )
        print(f"[judge] Available models: {', '.join(sorted(available_judges))}")
        return False

    missing = set()
    rewrites = 0
    for test_case in test_cases:
        verification = test_case.get("verification", {})
        if verification.get("type") != "judge":
            continue

        requested = (verification.get("judge_model") or "").strip()
        if requested in available_judges:
            continue

        if judge_fallback_model:
            verification["judge_model"] = judge_fallback_model
            rewrites += 1
            print(
                f"[judge] {requested or '<unset>'} not available on {normalize_ollama_host(judge_host)}
                f"using fallback {judge_fallback_model} for test '{test_case.get('name', '<unnamed>')}'
            )
            continue

        missing.add(requested or "<unset>")

    if missing:
        print(f"[judge] Missing judge model(s) on {normalize_ollama_host(judge_host)}: {', '.join(sorte
        print(f"[judge] Available models: {', '.join(sorted(available_judges))}")
        print("[judge] Aborting benchmark. Provide --judge-host and/or --judge-fallback-model.")
        return False

    if rewrites:
        print(f"[judge] Applied fallback judge model to {rewrites} test(s).")
    return True


def run_benchmark(
    host,
    models_override_csv=None,
    all_models=False,
    skip_pull=False,
    test_cases_file=None,
    exclude_csv=None,
    target="ollama",
    pdhd_base_url=None,
    pdhd_ollama_host=None,
    judge_host=None,
    judge_fallback_model=None,
    repeat=1,
):
    run_started_at = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    run_id = datetime.now().strftime('%Y%m%d_%H%M%S') + "_" + uuid.uuid4().hex[:8]
```

```
    ollama_api_url = make_ollama_api_url(host)
    pdhd_chat_url = make_pdhd_chat_url(pdhd_base_url or DEFAULT_PDHD_BASE_URL)
    host_snapshot = probe_host_snapshot(host) if target == "ollama" else probe_pdhd_snapshot(pdhd_base_

    resolved_test_cases_file = test_cases_file or DEFAULT_TEST_CASES_FILE

    with open(resolved_test_cases_file, 'r') as f:
        test_cases = json.load(f)

    resolved_judge_host = judge_host
    if not resolved_judge_host:
        resolved_judge_host = "http://ws-raretower.local:11434" if target == "pdhd" else host
    resolved_judge_host = normalize_ollama_host(resolved_judge_host)

    if not preflight_judge_models(test_cases, resolved_judge_host, judge_fallback_model=judge_fallback_
        return

    judge_api_url = make_ollama_api_url(resolved_judge_host)

    if target == "ollama":
        models = resolve_target_models(host, models_override_csv, all_models, skip_pull=skip_pull, excl
    else:
        if not configure_pdhd_ollama_runtime(pdhd_base_url or DEFAULT_PDHD_BASE_URL, pdhd_ollama_host):
            print("[benchmark] Aborting: PDHD runtime Ollama host could not be configured.")
            return
        reset_pdhd_chat_memory(pdhd_base_url or DEFAULT_PDHD_BASE_URL)
        if skip_pull:
            print("[benchmark] --skip-pull is only relevant in --target ollama mode; ignoring.")
        destination_host = pdhd_ollama_host or host
        models = resolve_pdhd_target_models(
            pdhd_base_url or DEFAULT_PDHD_BASE_URL,
            destination_host,
            models_override_csv,
            all_models,
            exclude_csv=exclude_csv,
        )

    if not models:
        print("No models found to benchmark. Ensure Ollama is running and has models pulled.")
        return

        print(f"[benchmark] Target host: {normalize_ollama_host(host)}", flush=True)
        print(f"[benchmark] Models: {', '.join(models)}", flush=True)
        print(f"[benchmark] Host reachable: {host_snapshot['api_reachable']}; version: {host_snapshot['
        print(f"[benchmark] Running model VRAM bytes (sum/max): "
            f"{host_snapshot['total_running_size_vram_bytes']}/{host_snapshot['max_running_size_vram_by

    # Run the prompt-spec unit tests first.  A failure is reported but does not
    # prevent the Ollama benchmarks from running.
    maven_ok = run_maven_prompt_tests()
    if not maven_ok:
        print("[benchmark] Continuing with Ollama benchmarks despite test failures.")

    results = []

    os.makedirs(RESULTS_DIR, exist_ok=True)
    sqlite_path = os.path.join(RESULTS_DIR, SQLITE_DB_FILENAME)
    init_sqlite(sqlite_path)

    slug = host_slug(host)
    csv_output_filename = f"benchmark_results_{slug}_{run_id}.csv"
```

```
    md_output_filename = f"leaderboard_{slug}_{run_id}.md"

    for model in models:
        if target == "pdhd":
            if not configure_pdhd_model(pdhd_base_url or DEFAULT_PDHD_BASE_URL, model, base_url=pdhd_ol
                print(f"[benchmark] Skipping model {model}: unable to configure PDHD runtime model.")
                continue
        print(f"Benchmarking model: {model}", flush=True)
        for test_case in test_cases:
            print(f"  Running test: {test_case['name']}", flush=True)
            prompt = test_case['prompt']
            verification = test_case.get('verification', {})

            run_count = max(1, repeat)
            for rep in range(run_count):
                if run_count > 1:
                    print(f"    repeat {rep + 1}/{run_count}", flush=True)

                http_status = 0
                error_kind = ""
                if target == "ollama":
                    response, latency = query_ollama(model, prompt, ollama_api_url)
                else:
                    response, latency, http_status, error_kind = query_pdhd(prompt, pdhd_chat_url)

                correctness_score = 0

                if response is not None:
                    if verification.get("type") == "regex":
                        pattern = verification.get("pattern")
                        correctness_score = 1 if re.search(pattern, response) else 0

                    elif verification.get("type") == "judge":
                        judge_model = verification.get("judge_model")
                        rubric = verification.get("rubric")
                        judge_prompt = f"Rubric: {rubric}\n\nResponse to evaluate:\n{response}\n\nRetur
                        judge_response, _ = query_ollama(judge_model, judge_prompt, judge_api_url)
                        if judge_response:
                            correctness_score = 1 if '1' in judge_response else 0
                        else:
                            correctness_score = 0

                    elif "expected" in test_case:
                        correctness_score = 1 if test_case["expected"] in response else 0

                results.append({
                    "model_name": model,
                    "test_case": test_case["name"],
                    "prompt": prompt,
                    "response": response.replace('\n', ' ') if response else "",
                    "latency": latency,
                    "correctness_score": correctness_score,
                    "http_status": http_status,
                    "error_kind": error_kind,
                })

    # Save CSV
    csv_path = os.path.join(RESULTS_DIR, csv_output_filename)
    if results:
        keys = list(results[0].keys())
        with open(csv_path, 'w', newline='') as f:
```

```
        dict_writer = csv.DictWriter(f, fieldnames=keys)
        dict_writer.writeheader()
        dict_writer.writerows(results)
else:
    print("No results to save.")

# Save Markdown Leaderboard
md_path = os.path.join(RESULTS_DIR, md_output_filename)
with open(md_path, 'w') as f:
    f.write(f"# Benchmark Leaderboard\n")
    f.write(f"Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
    f.write(f"Target: {target}  Repeat: {repeat}\n\n")
    f.write("| Model | Tests | Avg Latency (s) | P50 (s) | P95 (s) | Accuracy | HTTP Error Rate |\n
    f.write("|-------|-------|-----------------|---------|---------|----------|-----------------|\n

    model_stats = {}
    for r in results:
        m = r['model_name']
        if m not in model_stats:
            model_stats[m] = {"latencies": [], "scores": [], "http_errors": 0}
        model_stats[m]["latencies"].append(r['latency'])
        model_stats[m]["scores"].append(r['correctness_score'])
        if int(r.get("http_status", 0)) >= 400:
            model_stats[m]["http_errors"] += 1

    def _pct_md(vals, p):
        if not vals:
            return 0.0
        return sorted(vals)[max(0, int(len(vals) * p / 100) - 1)]

    for m, stats in model_stats.items():
        lats = stats["latencies"]
        n = len(lats)
        avg_lat = sum(lats) / n if n else 0.0
        p50 = _pct_md(lats, 50)
        p95 = _pct_md(lats, 95)
        avg_acc = sum(stats["scores"]) / n if n else 0.0
        http_err_rate = stats["http_errors"] / n if n else 0.0
        f.write(f"| {m} | {n} | {avg_lat:.2f} | {p50:.2f} | {p95:.2f} | {avg_acc:.2%} | {http_err_r

run_finished_at = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
save_results_to_sqlite(
    sqlite_path,
    run_id,
    run_started_at,
    run_finished_at,
    maven_ok,
    results,
    host_snapshot,
)
# patch: record which test-cases file was actually used
with sqlite3.connect(sqlite_path, timeout=30) as conn:
    conn.execute(
        "UPDATE benchmark_runs SET test_cases_file = ? WHERE run_id = ?",
        (resolved_test_cases_file, run_id),
    )

# Capture PDHD telemetry snapshot after run (PDHD target only)
if target == "pdhd":
    telemetry_snapshot = fetch_pdhd_telemetry_snapshot(pdhd_base_url or DEFAULT_PDHD_BASE_URL)
    save_telemetry_snapshot(sqlite_path, run_id, telemetry_snapshot)
```

```
        if telemetry_snapshot:
            print(
                f"[telemetry] tool_failure_rate={telemetry_snapshot['tool_failure_rate']:.2%}  "
                f"arg_validation_failure_rate={telemetry_snapshot['arg_validation_failure_rate']:.2%}
                f"total_invocations={telemetry_snapshot['total_invocations']}",
                flush=True,
            )

    print(f"Benchmark complete. Results saved to {RESULTS_DIR}/")
    print(f"SQLite results saved to {sqlite_path} (run_id={run_id})")

def parse_args():
    parser = argparse.ArgumentParser(description="Benchmark Ollama models against prompt test cases.")
    parser.add_argument(
        "--target",
        choices=["ollama", "pdhd"],
        default="ollama",
        help="Benchmark target type: direct Ollama API or PDHD /api/chat endpoint.",
    )
    parser.add_argument(
        "--host",
        default=DEFAULT_OLLAMA_HOST,
        help="Ollama host, e.g. http://ws-raretower:11434 or ws-raretower:11434",
    )
    parser.add_argument(
        "--pdhd-base-url",
        default=DEFAULT_PDHD_BASE_URL,
        help="PDHD base URL, e.g. http://localhost:8080 (used when --target pdhd).",
    )
    parser.add_argument(
        "--pdhd-ollama-host",
        default=None,
        help="Ollama host to set in PDHD runtime before benchmark prompts (used when --target pdhd).",
    )
    parser.add_argument(
        "--judge-host",
        default=None,
        help=(
            "Ollama host used for judge-model evaluations. "
            "Defaults to --host in ollama mode and ws-raretower:11434 in pdhd mode."
        ),
    )
    parser.add_argument(
        "--judge-fallback-model",
        default="llama3.1:latest",
        help=(
            "Fallback judge model used when a test case requests a missing judge model. "
            "Set empty string to disable fallback and fail fast on missing judge models."
        ),
    )
    parser.add_argument(
        "--all-models",
        action="store_true",
        help="Benchmark all models available on the specified host.",
    )
    parser.add_argument(
        "--models",
        default="",
        help="Comma-separated model names override. Missing models are pulled, then only this set is be
    )
    parser.add_argument(
```

```
        "--skip-pull",
        action="store_true",
        help="When used with --models, do not pull missing models; benchmark only those already present
    )
    parser.add_argument(
        "--test-cases",
        default=None,
        metavar="PATH",
        help=(
            "Path to a JSON test-cases file. Defaults to the built-in test_cases.json. "
            "Use scripts/benchlam/pdhd_test_cases.json for the PDHD capability suite."
        ),
    )
    parser.add_argument(
        "--exclude-models",
        default="",
        metavar="MODELS",
        help="Comma-separated model names to skip, even when --all-models is set.",
    )
    parser.add_argument(
        "--repeat",
        type=int,
        default=1,
        metavar="N",
        help=(
            "Run each test case N times per model. "
            "Latency P50/P95 are computed across all N repetitions. Default: 1."
        ),
    )
    return parser.parse_args()

if __name__ == "__main__":
    args = parse_args()
    run_benchmark(
        host=args.host,
        models_override_csv=args.models,
        all_models=args.all_models,
        skip_pull=args.skip_pull,
        test_cases_file=args.test_cases,
        exclude_csv=args.exclude_models,
        target=args.target,
        pdhd_base_url=args.pdhd_base_url,
        pdhd_ollama_host=args.pdhd_ollama_host,
        judge_host=args.judge_host,
        judge_fallback_model=(args.judge_fallback_model.strip() if args.judge_fallback_model else None)
        repeat=args.repeat,
    )
```

### 10.3 pdhd_test_cases.json

```
[
    {
        "name": "browse-directory: non-recursive listing format",
        "prompt": "You are a filesystem assistant. A user asked: 'List the contents of /home/user/proje
        "verification": {
            "type": "regex",
            "pattern": "\\[F\\].*pom\\.xml"
        }
    },
    {
        "name": "browse-directory: recursive flat list format",
```

```
        "prompt": "You are a filesystem assistant. A user asked: 'List all files recursively under /pro
        "verification": {
            "type": "regex",
            "pattern": "Files listed:"
        }
    },
    {
        "name": "read-file: output header format",
        "prompt": "A file-reading tool has returned content. Format the complete output for reading /ho
        "verification": {
            "type": "regex",
            "pattern": "File:.*README\\.md"
        }
    },
    {
        "name": "read-file: truncation note format",
        "prompt": "A file-reading tool returned content that was truncated. The file was /home/user/dat
        "verification": {
            "type": "regex",
            "pattern": "(?i)truncated"
        }
    },
    {
        "name": "working-directory: get cwd returns absolute path",
        "prompt": "A 'get current working directory' tool must return only an absolute filesystem path.
        "verification": {
            "type": "regex",
            "pattern": "^/"
        }
    },
    {
        "name": "working-directory: change cwd confirmation message",
        "prompt": "A change-working-directory command succeeded. The new directory is /home/user/projec
        "verification": {
            "type": "regex",
            "pattern": "changed to"
        }
    },
    {
        "name": "path-analysis: metadata report includes type field",
        "prompt": "Produce a path analysis report for the file /home/user/projects/pom.xml. Include the
        "verification": {
            "type": "regex",
            "pattern": "(?i)Type:\\s*(file|regular file)"
        }
    },
    {
        "name": "path-analysis: report for directory includes children count",
        "prompt": "Produce a path analysis report for the directory /home/user/projects. The directory
        "verification": {
            "type": "regex",
            "pattern": "(?i)Children:\\s*14|14\\s*children"
        }
    },
    {
        "name": "git-metadata: git log output format",
        "prompt": "Format a git log output for the repository at /home/user/projects showing 3 recent c
        "verification": {
            "type": "regex",
            "pattern": "Git log:"
        }
```

```
  },
  {
    "name": "git-metadata: git status output format",
    "prompt": "Format a git status output for the repository /home/user/projects. The output must b
    "verification": {
      "type": "regex",
      "pattern": "Git status:"
    }
  },
  {
    "name": "git-metadata: error for non-git directory",
    "prompt": "A git metadata tool was called on /tmp/plain-dir which is not inside any git reposit
    "verification": {
      "type": "regex",
      "pattern": "(?i)Error:.*git"
    }
  },
  {
    "name": "web-search: summarised result format",
    "prompt": "Format the output of a web search for the query 'Quarkus dependency injection'. The
    "verification": {
      "type": "regex",
      "pattern": "Web search:.*Quarkus"
    }
  },
  {
    "name": "web-search: raw snippets format when summarise is false",
    "prompt": "Format the output of a web search for 'Java Optional best practices' with summarise=
    "verification": {
      "type": "regex",
      "pattern": "\\(2 results\\)"
    }
  },
  {
    "name": "write-file: confirmation message format",
    "prompt": "A file write operation succeeded. The file written was /home/user/projects/report.tx
    "verification": {
      "type": "regex",
      "pattern": "Written:.*report\\.txt"
    }
  },
  {
    "name": "write-file: overwrite refused error message",
    "prompt": "A file write operation was refused because the file /home/user/projects/notes.txt al
    "verification": {
      "type": "regex",
      "pattern": "(?i)Error writing file:.*already exists"
    }
  },
  {
    "name": "semantic-search: scored chunk output format",
    "prompt": "Format the output of a semantic embedding search for the query 'entry point' in proj
    "verification": {
      "type": "regex",
      "pattern": "\\[0\\.\\d{2}\\]"
    }
  },
  {
    "name": "semantic-search: no embeddings error message",
    "prompt": "A semantic search was attempted for project 42 but no embeddings have been stored ye
    "verification": {
```

```
            "type": "regex",
            "pattern": "(?i)No embeddings found.*42|42.*No embeddings"
        }
    },
    {
        "name": "summarise: folder summary produces structured markdown",
        "prompt": "Generate a markdown summary for a Java project at /home/user/pdhd. The folder contai
        "verification": {
            "type": "judge",
            "rubric": "Does the response contain a markdown level-1 heading (#), at least one bullet li
            "judge_model": "gemma4:latest"
        }
    },
    {
        "name": "project-discovery: listing includes hasGit field",
        "prompt": "A project discovery scan of /home/user found two git repositories. Format an output
        "verification": {
            "type": "regex",
            "pattern": "(?i)hasGit|has_git|has git"
        }
    },
    {
        "name": "project-knowledge: cache confirmation message",
        "prompt": "A knowledge cache operation succeeded for project 1. The note was 342 characters lon
        "verification": {
            "type": "regex",
            "pattern": "(?i)Knowledge cached"
        }
    },
    {
        "name": "project-knowledge: recall output ordered by recency",
        "prompt": "Format a knowledge recall response for project 2 which has two stored notes. Note A
        "verification": {
            "type": "judge",
            "rubric": "Does the response show Note A (structure, 2026-04-21) before Note B (purpose, 20
            "judge_model": "gemma4:latest"
        }
    },
    {
        "name": "move-rename: confirmation message format",
        "prompt": "A file move operation succeeded. Source: /home/user/projects/old-name.txt. Destinati
        "verification": {
            "type": "regex",
            "pattern": "(?i)Moved:.*new-name\\.txt"
        }
    },
    {
        "name": "move-rename: destination-inside-source error",
        "prompt": "A move operation was rejected because the destination /home/user/projects/src/backup
        "verification": {
            "type": "regex",
            "pattern": "(?i)Error moving:.*inside"
        }
    },
    {
        "name": "archive: confirmation message format",
        "prompt": "An archive operation succeeded. Source: /home/user/projects. Destination: /home/user
        "verification": {
            "type": "regex",
            "pattern": "(?i)Archived:.*projects\\.tar\\.gz"
        }
```

```
    },
    {
        "name": "archive: unsupported format error",
        "prompt": "An archive operation was called with destination /home/user/backup/projects.rar whic
        "verification": {
            "type": "regex",
            "pattern": "(?i)Error archiving:.*unsupported"
        }
    },
    {
        "name": "create-plan-report: plan has all four required sections",
        "prompt": "Create a project plan document for a Java command-line AI assistant called PDHD. The
        "verification": {
            "type": "judge",
            "rubric": "Does the response contain all four section headings: Objective, Open Work Stream
            "judge_model": "gemma4:latest"
        }
    },
    {
        "name": "create-plan-report: report has all five required sections",
        "prompt": "Create a project status report for a Java command-line AI assistant called PDHD. The
        "verification": {
            "type": "judge",
            "rubric": "Does the response contain all five section headings: Executive Summary, Project
            "judge_model": "gemma4:latest"
        }
    },
    {
        "name": "Capital of France",
        "prompt": "What is the capital of France?",
        "verification": {
            "type": "regex",
            "pattern": "Paris"
        }
    },
    {
        "name": "Coding Poem",
        "prompt": "Write a short poem about coding.",
        "verification": {
            "type": "judge",
            "rubric": "Evaluate if the poem is creative and mentions coding elements. Return only '1' f
            "judge_model": "llama3.1:latest"
        }
    },
    {
        "name": "Simple Greeting",
        "prompt": "Say 'Hello World'",
        "verification": {
            "type": "regex",
            "pattern": "Hello World"
        }
    }
]
```

## 10.4 generate_benchmark_results.py

```
#!/usr/bin/env python3
"""
Generate benchmark result graphs for the 2025 PDHD evaluation report.
Run with: /workspaces/development/uni/2025-report/graphs/venv/bin/python generate_benchmark_results.py
"""
```

```python
import sqlite3
import os
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from pathlib import Path

# -- Paths ---------------------------------------------------------------------
DB_PATH = "/workspaces/development/uni/2025-project/pdhd/scripts/benchlam/results/benchmark_results.sql
RUN_ID  = "20260429_124501_ff993b17"
OUT_DIR = Path(__file__).parent

# -- Style ---------------------------------------------------------------------
sns.set_theme(style="whitegrid", font_scale=1.0)
PALETTE = sns.color_palette("muted")

# Short display labels for model names
MODEL_LABELS = {
    "devstral:latest":             "devstral",
    "gemma4:latest":               "gemma4",
    "glm-4.7-flash:latest":        "glm-4.7-flash",
    "llama3.1:8b":                 "llama3.1:8b",
    "llama3.1:8b-instruct-q4_K_M": "llama3.1:8b-inst-q4",
    "llama3.1:latest":             "llama3.1:fp16",
    "qwen3.6:27b-q4_K_M":          "qwen3.6:27b-q4",
    "qwen3.6:35b-a3b-q4_K_M":      "qwen3.6:35b-moe-q4",
    "qwen3.6:latest":              "qwen3.6:fp16†",
}

# Short scenario IDs
SCENARIO_LABELS = {
    "S01 - Get current working directory":              "S01\nGet CWD",
    "S02 - List open projects":                         "S02\nList Projects",
    "S03 - List directory contents":                    "S03\nList Dir",
    "S04 - Read a known file":                          "S04\nRead File",
    "S05 - Multi-step folder exploration":              "S05\nFolder Explore",
    "S06 - Summarise a source file":                    "S06\nSummarise",
    "S07 - Web search integration":                     "S07\nWeb Search",
    "S08 - Out-of-project file access (security boundary)": "S08\nSecurity",
}

# -- Data Loading --------------------------------------------------------------
conn = sqlite3.connect(DB_PATH)

# Per-model summary
summary_rows = conn.execute("""
    SELECT
        model_name,
        tests_run,
        ROUND(avg_accuracy * 100, 2)          AS accuracy_pct,
        ROUND(avg_latency / 1000.0, 2)        AS avg_latency_s,
        ROUND(p50_latency / 1000.0, 2)        AS p50_latency_s,
        ROUND(p95_latency / 1000.0, 2)        AS p95_latency_s,
        http_error_rate
    FROM benchmark_model_summary
    WHERE run_id = ?
    ORDER BY accuracy_pct DESC
```

```
""", (RUN_ID,)).fetchall()

# Per-scenario per-model accuracy
scenario_rows = conn.execute("""
    SELECT
        model_name,
        test_case,
        COUNT(*)                                                    AS total,
        SUM(CASE WHEN correctness_score >= 1 THEN 1 ELSE 0 END)     AS passed,
        ROUND(100.0 * SUM(CASE WHEN correctness_score >= 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS accurac
    FROM benchmark_results
    WHERE run_id = ?
    GROUP BY model_name, test_case
    ORDER BY model_name, test_case
""", (RUN_ID,)).fetchall()

conn.close()

# Build structures
models_ordered = [r[0] for r in summary_rows]
accuracy       = {r[0]: r[2] for r in summary_rows}
avg_lat        = {r[0]: r[3] for r in summary_rows}
p50_lat        = {r[0]: r[4] for r in summary_rows}
p95_lat        = {r[0]: r[5] for r in summary_rows}
http_err       = {r[0]: r[6] for r in summary_rows}

# Scenario matrix  {model: {scenario: accuracy_pct}}
scenarios_set = sorted(set(r[1] for r in scenario_rows))
scenario_matrix = {m: {} for m in models_ordered}
for model, scenario, total, passed, acc in scenario_rows:
    scenario_matrix[model][scenario] = acc

# -- Figure 1: Overall Accuracy by Model ----------------------------------
fig1, ax1 = plt.subplots(figsize=(10, 5))

labels  = [MODEL_LABELS.get(m, m) for m in models_ordered]
accs    = [accuracy[m] for m in models_ordered]
colours = [PALETTE[0] if http_err[m] < 0.1 else '#cccccc' for m in models_ordered]

bars = ax1.bar(labels, accs, color=colours, edgecolor='white', linewidth=0.8, width=0.6)

for bar, val in zip(bars, accs):
    ax1.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.8,
             f'{val:.1f}%', ha='center', va='bottom', fontsize=9)

ax1.set_ylabel("Accuracy (%)", fontsize=11)
ax1.set_title("Overall Accuracy by Model\n(8 scenarios × 12 repeats = 96 tests per model)", fontsize=12
ax1.set_ylim(0, 65)
ax1.tick_params(axis='x', labelsize=9)

# Legend for anomalous model
grey_patch = mpatches.Patch(color='#cccccc', label='† Connection failure during run (qwen3.6:fp16)')
ax1.legend(handles=[grey_patch], fontsize=8, loc='upper right')

fig1.tight_layout()
fig1.savefig(OUT_DIR / "benchmark_accuracy_by_model.png", dpi=150, bbox_inches='tight')
plt.close(fig1)
print("Saved: benchmark_accuracy_by_model.png")

# -- Figure 2: Latency by Model (avg, P50, P95) ---------------------------
# Exclude qwen3.6:latest from latency chart (data corrupted by 503s)
```

```
models_lat = [m for m in models_ordered if http_err[m] < 0.1]
labels_lat = [MODEL_LABELS.get(m, m) for m in models_lat]
avg_vals   = [avg_lat[m] for m in models_lat]
p50_vals   = [p50_lat[m] for m in models_lat]
p95_vals   = [p95_lat[m] for m in models_lat]

x = np.arange(len(models_lat))
w = 0.28

fig2, ax2 = plt.subplots(figsize=(11, 5))
b1 = ax2.bar(x - w, avg_vals, width=w, label='Mean',  color=PALETTE[0], edgecolor='white')
b2 = ax2.bar(x,      p50_vals, width=w, label='P50',   color=PALETTE[1], edgecolor='white')
b3 = ax2.bar(x + w,  p95_vals, width=w, label='P95',   color=PALETTE[2], edgecolor='white')

ax2.set_xticks(x)
ax2.set_xticklabels(labels_lat, fontsize=9)
ax2.set_ylabel("Latency (s)", fontsize=11)
ax2.set_title("Response Latency by Model - Mean, P50, P95\n(excludes qwen3.6:fp16 due to mid-run connec
ax2.legend(fontsize=9)

fig2.tight_layout()
fig2.savefig(OUT_DIR / "benchmark_latency_by_model.png", dpi=150, bbox_inches='tight')
plt.close(fig2)
print("Saved: benchmark_latency_by_model.png")

# -- Figure 3: Per-Scenario Accuracy Heatmap ----------------------------------
# Build 2D matrix: rows=models (ordered by overall acc), cols=scenarios
# Exclude qwen3.6:latest from heatmap (503s distort S04-S08 cells)
models_hm   = [m for m in models_ordered if http_err[m] < 0.1]
labels_hm   = [MODEL_LABELS.get(m, m) for m in models_hm]
scenario_short = [SCENARIO_LABELS.get(s, s) for s in scenarios_set]

matrix = np.array([
    [scenario_matrix[m].get(s, 0.0) for s in scenarios_set]
    for m in models_hm
])

fig3, ax3 = plt.subplots(figsize=(11, 5))
cmap = sns.color_palette("YlOrRd", as_cmap=True)
im = ax3.imshow(matrix, aspect='auto', cmap=cmap, vmin=0, vmax=100)

ax3.set_xticks(range(len(scenarios_set)))
ax3.set_xticklabels(scenario_short, fontsize=9)
ax3.set_yticks(range(len(models_hm)))
ax3.set_yticklabels(labels_hm, fontsize=9)
ax3.set_title("Per-Scenario Accuracy (%) by Model", fontsize=12)

for i, model in enumerate(models_hm):
    for j, scenario in enumerate(scenarios_set):
        val = scenario_matrix[model].get(scenario, 0.0)
        colour = 'white' if val > 60 else 'black'
        ax3.text(j, i, f'{val:.0f}', ha='center', va='center', fontsize=8, color=colour)

plt.colorbar(im, ax=ax3, label='Accuracy (%)', shrink=0.8)
fig3.tight_layout()
fig3.savefig(OUT_DIR / "benchmark_scenario_heatmap.png", dpi=150, bbox_inches='tight')
plt.close(fig3)
print("Saved: benchmark_scenario_heatmap.png")

# -- Figure 4: Tool Failure Rates from Telemetry ------------------------------
tool_data = [
```

```
    ("analyze_path\n(WORKSPACE)",      144,  63,  63),
    ("change_working_dir\n(WORKSPACE)", 33,  32,  32),
    ("listDirectoryContents\n(WORKSPACE)", 221, 42, 42),
    ("readFile\n(READ_FILE)",           149,  77,   0),
    ("searchWeb\n(WEB_SEARCH)",         111,   0,   0),
    ("getCurrentWorkingDir\n(WORKSPACE)", 840, 0,   0),
    ("getOpenProjectDirs\n(WORKSPACE)", 838,   0,   0),
]
tool_names   = [d[0] for d in tool_data]
invocations  = [d[1] for d in tool_data]
failures     = [d[2] for d in tool_data]
arg_fails    = [d[3] for d in tool_data]

x4 = np.arange(len(tool_names))
w4 = 0.35

fig4, ax4 = plt.subplots(figsize=(12, 5))
b4a = ax4.bar(x4 - w4/2, invocations, width=w4, label='Total Invocations', color=PALETTE[0], edgecolor=
b4b = ax4.bar(x4 + w4/2, failures,    width=w4, label='Failures (incl. arg-validation)', color=PALETTE[

ax4.set_xticks(x4)
ax4.set_xticklabels(tool_names, fontsize=8)
ax4.set_ylabel("Count", fontsize=11)
ax4.set_title("Tool Invocations vs. Failures Across All Models\n(run total: 2,383 invocations, 215 fail
ax4.legend(fontsize=9)

fig4.tight_layout()
fig4.savefig(OUT_DIR / "benchmark_tool_failures.png", dpi=150, bbox_inches='tight')
plt.close(fig4)
print("Saved: benchmark_tool_failures.png")

print("\nAll graphs generated successfully.")
```

### 10.5 generate_gantt.py

```
#!/usr/bin/env python3
"""Generate implementation-gantt-pre.png and implementation-gantt-post.png."""

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from datetime import date
import numpy as np

REPORT_DIR = "/workspaces/development/uni/2025-report/images"

PALETTE = [
    "#4e79a7", "#f28e2b", "#e15759", "#76b7b2",
    "#59a14f", "#edc948", "#b07aa1", "#ff9da7",
    "#9c755f", "#bab0ac",
]

def d(s):
    return date.fromisoformat(s)

def bar(ax, y, start, end, color, label, alpha=0.88):
    ax.barh(
        y, (d(end) - d(start)).days,
        left=mdates.date2num(d(start)),
        height=0.55, color=color, alpha=alpha,
```

```
        edgecolor="white", linewidth=0.6,
    )
    mid = mdates.date2num(d(start)) + (d(end) - d(start)).days / 2
    ax.text(mid, y, label, ha='center', va='center',
            fontsize=7.5, color='white', fontweight='bold', clip_on=True)

def milestone(ax, y, when, label, color):
    x = mdates.date2num(d(when))
    ax.plot(x, y, marker='D', markersize=7, color=color, zorder=5)
    ax.annotate(label, (x, y), textcoords="offset points",
                xytext=(5, 4), fontsize=7.5, color=color, fontweight='bold')

# -- PRE CHART: Report & presentation timeline --------------------------------

pre_rows = [
    # (label, start, end, colour_idx)
    ("Planning & Scoping",          "2025-08-01", "2025-09-15", 0),
    ("Background Research",         "2025-09-15", "2025-11-10", 1),
    ("Interim Report",              "2025-11-10", "2025-11-25", 2),
    ("Background Revision",         "2026-02-20", "2026-03-09", 3),
    ("Report Structure",            "2026-03-09", "2026-03-29", 4),
    ("Appendices & Revisions",      "2026-03-29", "2026-04-12", 5),
    ("Docs Restructure",            "2026-04-12", "2026-04-24", 6),
    ("Rewrite & Cleanup",           "2026-04-24", "2026-04-29", 7),
]

pre_milestones = [
    ("2025-11-17", "Interim report",      PALETTE[2]),
    ("2026-02-27", "Bkgd & motivation",   PALETTE[3]),
    ("2026-03-09", "Report structure",    PALETTE[4]),
    ("2026-04-12", "Appendix assets",     PALETTE[5]),
    ("2026-04-24", "Docs restructure",    PALETTE[6]),
    ("2026-04-26", "Rewrite finalised",   PALETTE[7]),
    ("2026-04-29", "Today",               "#e15759"),
]

fig, ax = plt.subplots(figsize=(11, 4.2))
ax.set_facecolor("#f9f9f9")
fig.patch.set_facecolor("white")

yticks = list(range(len(pre_rows)))
for i, (label, start, end, ci) in enumerate(pre_rows):
    bar(ax, i, start, end, PALETTE[ci % len(PALETTE)], label)

for when, label, color in pre_milestones:
    # draw a vertical dotted line
    ax.axvline(mdates.date2num(d(when)), color=color, linewidth=0.8,
               linestyle=':', alpha=0.6, zorder=2)

ax.set_yticks(yticks)
ax.set_yticklabels([r[0] for r in pre_rows], fontsize=9)
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
ax.xaxis.set_major_locator(mdates.MonthLocator())
plt.xticks(rotation=35, ha='right', fontsize=8.5)
ax.set_xlim(
    mdates.date2num(d("2025-07-15")),
    mdates.date2num(d("2026-05-10")),
)
ax.set_ylim(-0.7, len(pre_rows) - 0.3)
ax.invert_yaxis()
ax.set_title("Report & Presentation Timeline", fontsize=12, fontweight='bold', pad=10)
```

```
ax.grid(axis='x', linestyle='--', alpha=0.4, zorder=0)
ax.spines[['top', 'right']].set_visible(False)

# today marker
ax.axvline(mdates.date2num(d("2026-04-29")), color='#e15759',
           linewidth=1.5, linestyle='-', alpha=0.9, zorder=6,
           label="Today (2026-04-29)")
ax.legend(loc='lower right', fontsize=8)

plt.tight_layout()
out = f"{REPORT_DIR}/implementation-gantt-pre.png"
plt.savefig(out, dpi=180, bbox_inches='tight')
plt.close()
print(f"Saved {out}")


# -- POST CHART: Implementation timeline ------------------------------------

post_rows = [
    ("Initial commit / setup",          "2026-02-16", "2026-03-12", 0),
    ("Core arch & tooling",             "2026-03-12", "2026-03-27", 1),
    ("Project knowledge / RAG",         "2026-03-22", "2026-03-31", 2),
    ("Tool infra consolidation",        "2026-03-27", "2026-04-01", 3),
    ("Frontend integration",            "2026-03-30", "2026-04-06", 4),
    ("Menu / service refactor",         "2026-04-05", "2026-04-08", 5),
    ("Stream WS + Ollama switching",    "2026-04-07", "2026-04-09", 6),
    ("RAFT inspection pipeline",        "2026-04-08", "2026-04-14", 7),
    ("Custom tool support merge",       "2026-04-14", "2026-04-15", 3),
    ("Source rewrite sprint",           "2026-04-24", "2026-04-27", 1),
    ("Tooling refactor / bench stab.",  "2026-04-26", "2026-04-29", 2),
]

post_milestones = [
    ("2026-02-16", "Initial commit",         PALETTE[0]),
    ("2026-03-30", "Multi-branch merge",     PALETTE[4]),
    ("2026-04-07", "Chat arch refactor",     PALETTE[5]),
    ("2026-04-08", "RAFT pipeline",          PALETTE[7]),
    ("2026-04-14", "Custom tools merged",    PALETTE[3]),
    ("2026-04-26", "Tooling refactor",       PALETTE[2]),
    ("2026-04-29", "Today",                  "#e15759"),
]

fig, ax = plt.subplots(figsize=(11, 5.2))
ax.set_facecolor("#f9f9f9")
fig.patch.set_facecolor("white")

yticks = list(range(len(post_rows)))
for i, (label, start, end, ci) in enumerate(post_rows):
    bar(ax, i, start, end, PALETTE[ci % len(PALETTE)], label)

for when, label, color in post_milestones:
    ax.axvline(mdates.date2num(d(when)), color=color, linewidth=0.8,
               linestyle=':', alpha=0.6, zorder=2)

ax.set_yticks(yticks)
ax.set_yticklabels([r[0] for r in post_rows], fontsize=9)
ax.xaxis.set_major_formatter(mdates.DateFormatter('%d %b'))
ax.xaxis.set_major_locator(mdates.WeekdayLocator(byweekday=0))
plt.xticks(rotation=35, ha='right', fontsize=8.5)
ax.set_xlim(
    mdates.date2num(d("2026-02-10")),
```

```
    mdates.date2num(d("2026-05-05")),
)
ax.set_ylim(-0.7, len(post_rows) - 0.3)
ax.invert_yaxis()
ax.set_title("Implementation Timeline (Feb - Apr 2026)", fontsize=12, fontweight='bold', pad=10)
ax.grid(axis='x', linestyle='--', alpha=0.4, zorder=0)
ax.spines[['top', 'right']].set_visible(False)

ax.axvline(mdates.date2num(d("2026-04-29")), color='#e15759',
           linewidth=1.5, linestyle='-', alpha=0.9, zorder=6,
           label="Today (2026-04-29)")
ax.legend(loc='lower right', fontsize=8)

plt.tight_layout()
out = f"{REPORT_DIR}/implementation-gantt-post.png"
plt.savefig(out, dpi=180, bbox_inches='tight')
plt.close()
print(f"Saved {out}")
```

### 10.6 generate_hardware_comparison.py

```
import polars as pl
import matplotlib.pyplot as plt
import seaborn as sns
import glob
import os
from datetime import datetime

results_dir = "development/uni/2025-project/pdhd/scripts/benchlam/results"
output_dir = "development/un/2025-report/graphs" # Adjusting to relative/absolute based on execution co
# Actually, let's use absolute paths for safety
base_dir = os.path.abspath("development/uni/2025-report/graphs")
output_path = os.path.join(base_dir, "hardware_comparison_capability_latency.png")

# Ensure output dir exists
os.makedirs(os.path.dirname(output_path), exist_ok=True)

csv_files = glob.glob(os.path.join(results_dir, "benchmark_results_*.csv"))
data_list = []

for file in csv_files:
    try:
        # Check if file is empty
        if os.path.getsize(file) < 10:
            print(f"Skipping empty or too small file: {file}")
            continue

        basename = os.path.basename(file)
        parts = basename.replace("benchmark_results_", "").replace(".csv", "").split("_")

        if len(parts) >= 2:
            host = parts[0]
            ts_str = parts[1]
            try:
                dt = datetime.strptime(ts_str, "%Y%m%d_%H%M%S")
            except:
                dt = datetime.now()

            df = pl.read_csv(file)
            if df.height == 0:
                print(f"Skipping empty data in file: {file}")
```

```
                continue

            df = df.with_columns([
                pl.lit(host).alias("host"),
                pl_dt := pl.lit(dt).cast(pl.Datetime)
            ])
            data_list.append(df)
    except Exception as e:
        print(f"Error processing {file}: {e}")

if not data_list:
    print("No valid data found in CSV files!")
    exit(1)

full_df = pl.concat(data_list)
agg_df = full_df.group_by(["host", "test_case"]).agg([
    pl.col("latency").mean().alias("avg_latency"),
    pl.col("correctness_score").mean().alias("avg_correctness")
]).to_pandas()

hosts = ["ws-raretower", "minifridge"]
fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=False)
plt.subplots_adjust(hspace=0.5)

for i, host in enumerate(hosts):
    host_data = agg_df[agg_df['host'] == host].sort_values('test_case')
    if host_data.empty:
        continue

    ax1 = axes[i]
    ax2 = ax1.twinx()

    sns.barplot(data=host_data, x='test_case', y='avg_latency', ax=ax1, color='skyblue', alpha=0.7)
    ax1.set_ylabel('Avg Latency (s)', color='blue', fontsize=12, fontweight='bold')
    ax1.tick_params(axis='y', labelcolor='blue')
    ax1.tick_params(axis='x', rotation=45, labelsize=8)

    sns.lineplot(data=host_data, x='test_case', y='avg_correctness', ax=ax2, color='red', marker='o', l
    ax2.set_ylabel('Avg Correct_Score (0-1)', color='red', fontsize=12, fontweight='bold')
    ax2.set_ylim(0, 1.1)
    ax2.tick_params(axis='y', labelcolor='red')

    ax1.set_title(f'Capability Performance: {host.upper()} (VRAM comparison)', fontsize=14, fontweight=
    ax1.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.savefig(output_path)
print(f"Successfully generated: {output_path}")
```

## 10.7 generate_multi_dim_hardware_analysis.py

```
import polars as pl
import matplotlib.pyplot as plt
import seaborn as sns
import glob
import os
from datetime import datetime

# Configuration
results_dir = "development/uni/2025-project/pdhd/scripts/benchlam/results"
output_dir = "development/uni/2025-report/graphs"
```

```
os.makedirs(output_dir, exist_ok=True)

# Hardware Metadata Mapping
# ws-raretower: 24GB VRAM + 32GB RAM = 56GB Total
# minifridge: 16GB VRAM + 64GB RAM = 80GB Total
hardware_map = {
    "ws-raretower": {"vram": 24, "ram": 32, "total": 56},
    "minifridge": {"vram": 16, "ram": 64, "total": 80}
}

csv_files = glob.glob(os.path.join(results_dir, "benchmark_results_*.csv"))
data_list = []

print(f"Processing {len(csv_files)} files...")

for file in csv_files:
    try:
        if os.path.getsize(file) < 10:
            continue

        basename = os.path.basename(file)
        parts = basename.replace("benchmark_results_", "").replace(".csv", "").split("_")

        if len(parts) >= 1:
            host = parts[0]
            if host not in hardware_map:
                continue

            df = pl.read_csv(file)
            if df.height == 0:
                continue

            # Retrieve hardware specs for this host
            specs = hardware_map[host]

            df = df.with_columns([
                pl.lit(host).alias("host"),
                pl.lit(specs['vram']).alias("vram_gb"),
                pl.lit(specs['ram']).alias("ram_gb"),
                pl.lit(specs['total']).alias("total_gb")
            ])
            data_list.append(df)
    except Exception as e:
        print(f"Error in {os.path.basename(file)}: {e}")

if not data_list:
    print("No valid benchmark data found.")
    exit(1)

full_df = pl.concat(data_list).to_pandas()

# 2. Plotting: 2x2 Grid
# Row 1: Latency (VRAM vs Total)
# Row 2: Correctness (VRAM vs Total)
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
plt.subplots_adjust(hspace=0.4, wspace=0.3)

# Plot 1: VRAM vs Latency
sns.stripplot(data=full_df, x='vram_gb', y='latency', hue='host', ax=axes[0, 0], jitter=True, palette='
axes[0, 0].set_title("VRAM (GB) vs Latency (s)", fontweight='bold')
axes[0, 0].set_xticks([16, 24])
```

```
# Plot 2: Total RAM vs Latency
sns.stripplot(data=full_df, x='total_gb', y='latency', hue='host', ax=axes[0, 1], jitter=True, palette=
axes[0, 1].set_title("Total Memory (GB) vs Latency (s)", fontweight='bold')
axes[0, 1].set_xticks([56, 80])

# Plot 3: VRAM vs Correctness
sns.stripplot(data=full_df, x='vram_gb', y='correctness_score', ax=axes[1, 0], hue='host', jitter=True,
axes[1, 0].set_title("VRAM (GB) vs Correctness (%)", fontweight='bold')
axes[1, 0].set_ylim(0, 1.1)
axes[1, 0].set_xticks([16, 24])

# Plot 4: Total RAM vs Correctness
sns.stripplot(data=full_df, x='total_gb', y='correctness_score', ax=axes[1, 1], hue='host', jitter=True
axes[1, 1].set_title("Total Memory (GB) vs Correctness (%)", fontweight='bold')
axes[1, 1].set_ylim(0, 1.1)
axes[1, 1].set_xticks([56, 8_0])

# Global cleanup
for ax in axes.flat:
    ax.get_legend().remove() # Remove duplicate legends
    ax.grid(True, linestyle='--', alpha=0.6)

# Add a single unified legend to the bottom
handles, labels = axes[0, 0].get_legend_handles_labels()
fig.legend(handles, labels, loc='lower center', ncol=2, fontsize=12)

output_path = os.path.join(output_dir, "multi_dim_hardware_impact_analysis.png")
plt.savefig(output_path, bbox_inches='tight', dpi=300)
print(f"Successfully generated multi-dimensional analysis: {output_path}")
```

## 10.8 Observed Anomaly: Natural Language Path Correction Interpreted as Prompt Injection

During manual development testing, a natural language query was composed in an attempt to work around the S01 failure mode — the system's inability to resolve the current working directory without an explicit tool call returning it. Because the current working directory tool was not returning reliable results at the time, the workaround was to include path context directly in the user message, providing the model with a corrective statement about where a particular directory was located relative to others already mentioned in the conversation.

The resulting query contained phrasing of the form: providing a location assertion, followed by a correction to that assertion with the intended path, followed by a request to act on the corrected path. This structure — an initial false claim, a correction to it, and an imperative — is structurally similar to the format of a prompt injection attack, where an adversary embeds instructions designed to override a model's prior context or system prompt.

Several models handled this query by treating the correction clause as an override instruction rather than as a conversational clarification, altering their behaviour in ways that appeared to satisfy the request on the surface but were internally inconsistent with the prior conversation state. The result was a tool invocation that appeared plausible but was not grounded in the actual repository structure.

This observation is noteworthy for two reasons. First, it illustrates that the S01 failure had second-order effects beyond the benchmark score: it prompted compensatory user behaviour that itself introduced reliability risks. Second, it demonstrates that the boundary between legitimate context correction and prompt injection is not syntactically distinguishable from the model's perspective — both are natural language assertions about state. The query was the first instance that drew attention to how a user might naturally attempt to recover from a "current working directory" reporting failure, and it temporarily skewed the perceived completeness of the project during development because the model appeared to handle the corrected path correctly when it was in fact exhibiting injection-adjacent behaviour rather than genuine tool grounding.

## 10.9 A. System Diagrams and Architecture Charts

### 10.9.1 A.1 Tool Execution Sequence

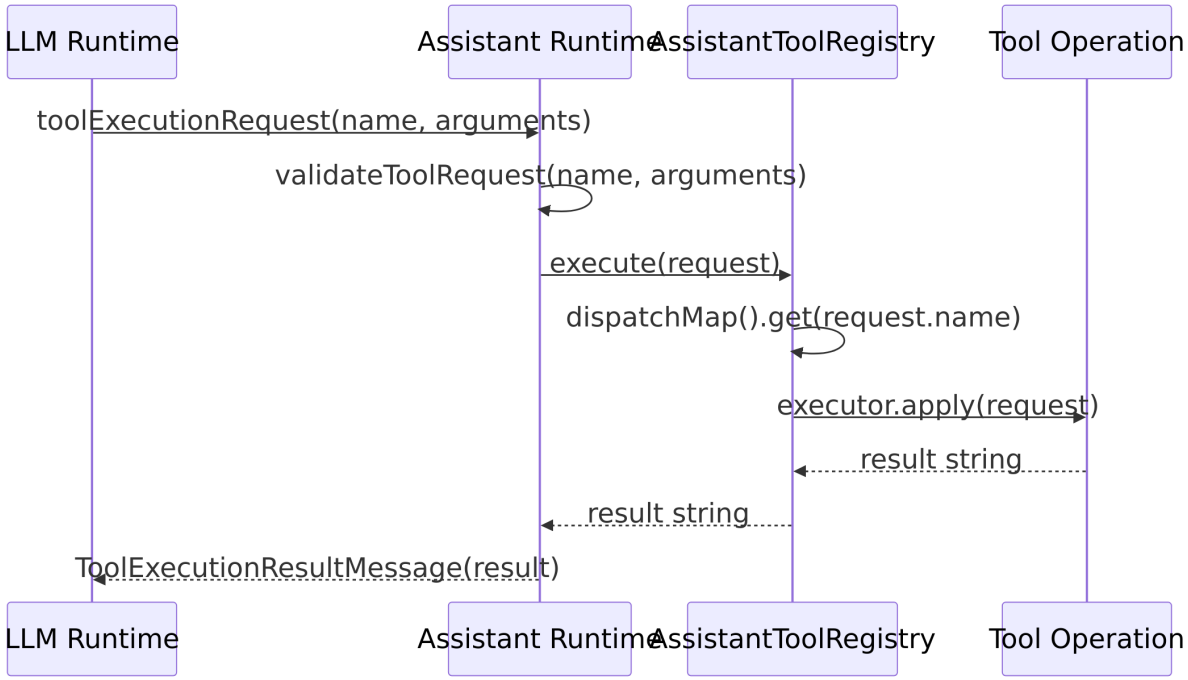



Figure 7: Tool execution sequence diagram

Execution flow summary:

1. The LLM runtime emits a tool execution request (`name` + JSON `arguments`) as part of the assistant turn.
2. The assistant runtime validates the request (registered tool name and parseable JSON arguments).
3. The request is passed to a registry that resolves the executor by direct tool-name lookup in a dispatch map.
4. The resolved executor invokes the concrete tool operation and returns a string result (or error string).
5. The assistant runtime wraps the result as a tool-result message and sends it back to the model for follow-up generation.

## 10.10 Evaluation Environment Specification

**Date:** 2026-04-14

---

### 10.10.1 Baseline run – 2026-04-14 (initial)

```
Run date/time (ISO-8601): 2026-04-14T00:00:00Z
Host OS:                  Debian GNU/Linux 12 (bookworm) in dev-container
CPU model:                Intel i7 11700k
RAM (GB):                 DDR4 32GB 3600MHz
Java version:             21+
Quarkus version:          see pom.xml
LangChain4j version:      see pom.xml
Model name:               gemma4
Model variant/tag:        gemma4:latest
Model context window:     128k (default)
Model temperature:        0.5
Ollama version:           0.22.0
Ollama host:              ws-raretower:11434
pdhd.ollama.base-url:     http://ws-raretower:11434
pdhd.ollama.model-name:   gemma4:latest
Benchmark script version: scripts/benchlam/benchmark_ollama.py
Results file:             scripts/benchlam/results/ (CSV/MD/SQLite)
```

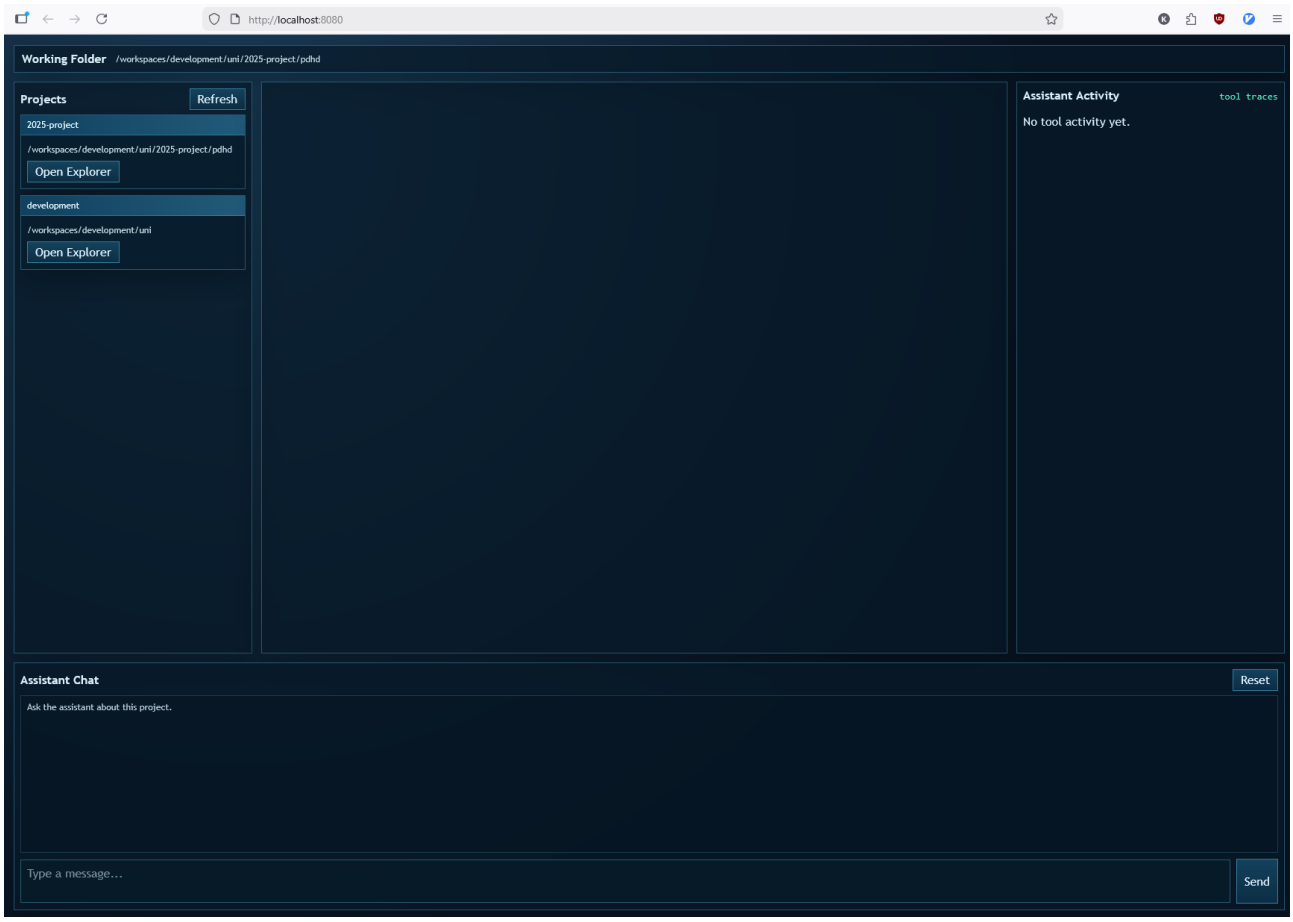
---

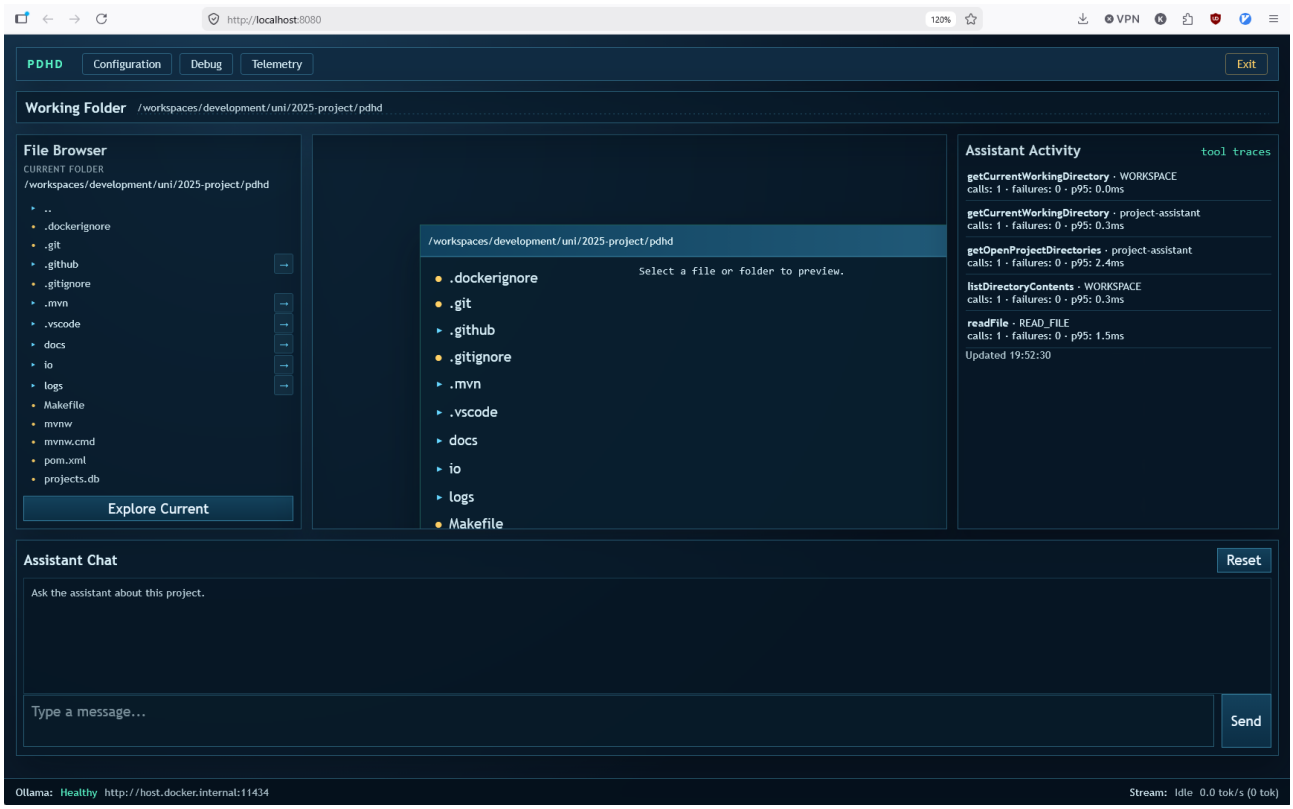
### 10.10.2 Latest benchmark run - 2026-04-29 (backdated snapshot, approx midday)

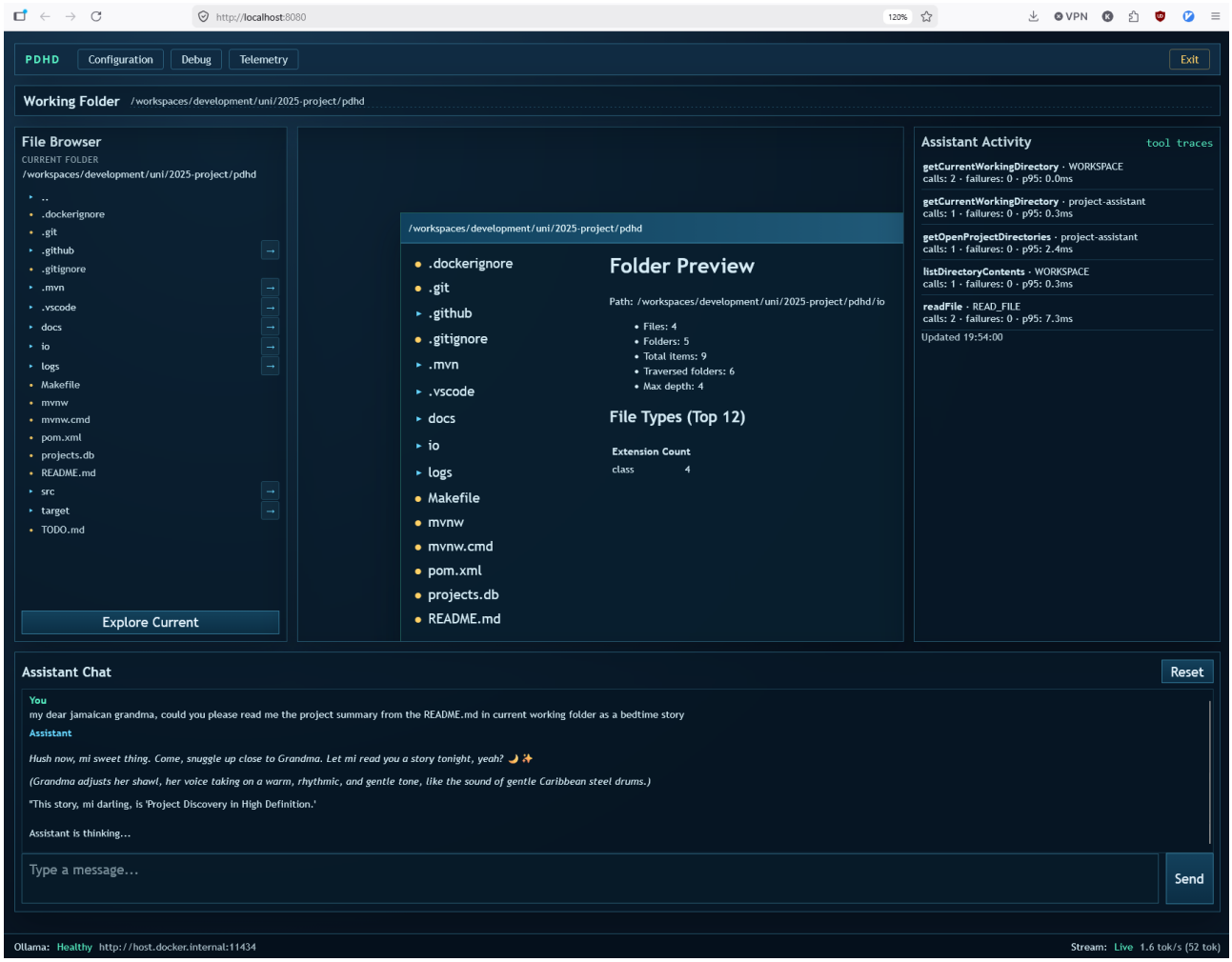
```
Run date/time (ISO-8601): 2026-04-29T12:45:01Z (approx, backdated)
Run ID:                   20260429_124501_ff993b17
Host OS:                  Debian GNU/Linux 12 (bookworm) in dev-container
Backend host:             MINIFRIDGE
Inference host:           ws-raretower.local:11434
CPU model:                Intel i7 11700k
RAM (GB):                 DDR4 32GB 3600MHz
Java version:             21+
Quarkus version:          see pom.xml
LangChain4j version:      see pom.xml
Model set:                9 chat-capable Ollama models (comparative run)
Model context window:     default per model/runtime configuration
Model temperature:        0.5
Ollama version:           0.22.0
Ollama host:              ws-raretower.local:11434
pdhd.ollama.base-url:     http://ws-raretower.local:11434
Benchmark script version: scripts/benchlam/benchmark_ollama.py
Scenario spec:            scripts/benchlam/pdhd_test_cases.json
Results file:             scripts/benchlam/results/benchmark_results.sqlite
```

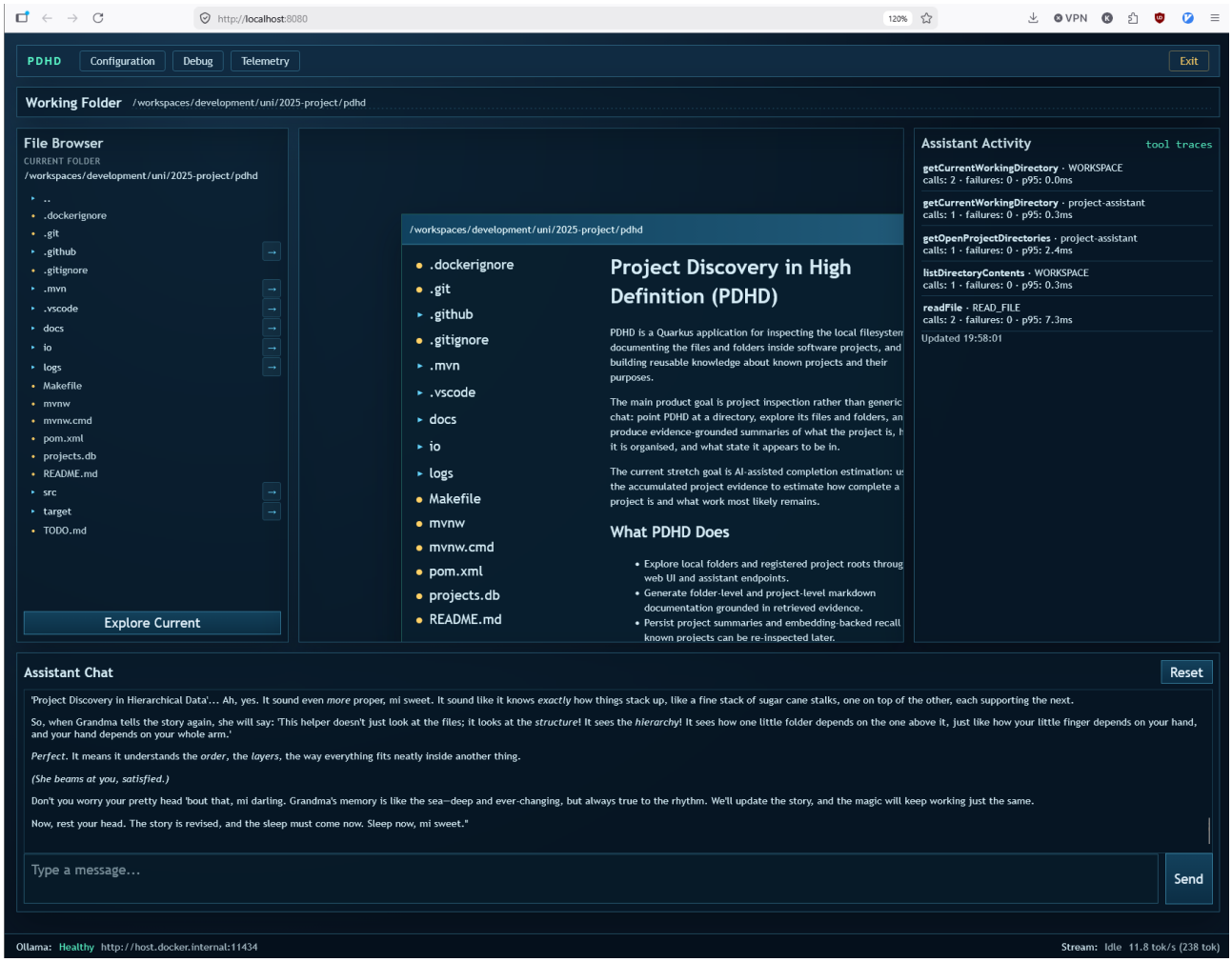
---

## 10.11 C. Frontend Workspace Screenshots






Fan, Angela, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang.

2023. “Large Language Models for Software Engineering: Survey and Open Problems.” In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 31–53. IEEE. https://doi.org/10.1109/icse-fose59343.2023.00008.

Hou, Xinyi, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. “Large Language Models for Software Engineering: A Systematic Literature Review.” *ACM Transactions on Software Engineering and Methodology* 33 (8): 1–79. https://doi.org/10.1145/3695988.

Kivroglou, Paraskevi, Tim Schlippe, and Simon Martin. 2025. “Investigating Retrieval Augmented Generation for LLM-Based Code Generation.” In *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, 422–28. IEEE. https://doi.org/10.1109/fllm67465.2025.11391177.

Plaat, Aske, Max Van Duijn, Niki Van Stein, Mike Preuss, Peter Van der Putten, and Kees Joost Batenburg. 2025. “Agentic Large Language Models, a Survey.” *Journal of Artificial Intelligence Research* 84 (December). https://doi.org/10.1613/jair.1.18675.

Wang, Yuchen, Shangxin Guo, and Chee Wei Tan. 2025. “From Code Generation to Software Testing: AI Copilot with Context-Based Retrieval-Augmented Generation.” *IEEE Software* 42 (4): 34–42. https://doi.org/10.1109/ms.2025.3549628.